TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH 2

XÂY DỰNG ĐỒ THỊ TRI THỨC DINH DƯỠNG BỆNH NHÂN

LÊ QUANG HUY – 223571

Ngành Kỹ Thuật Phần Mềm

Mã số ngành 7480103

Cần Thơ, 28 tháng 3 năm 2026

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH 2

XÂY DỰNG ĐỒ THỊ TRI THỨC DINH DƯỠNG BỆNH NHÂN

Ngành Kỹ Thuật Phần Mềm

Mã số ngành 7480103

Giảng viên hướng dẫn

Giảng viên: Trần Văn Thiện

Sinh viên thực hiện

Lê Quang Huy – 223571

Cần Thơ, tháng 3 năm 2026

LỜI CẢM ƠN

Với lòng biết ơn sâu sắc cho phép em gửi lời cảm ơn chân thành nhất tới:

Trường Đại học Nam Cần Thơ, Khoa Công Nghệ Thông Tin, cùng các giảng viên đã tận tình chỉ dạy và tạo điều kiện giúp đỡ em trong quá trình học tập, nghiên cứu và hoàn thành đồ án.

Đặc biệt em xin bày tỏ lòng biết ơn sâu sắc đến Thầy Trần Văn Thiện – người hướng dẫn và cũng là người đã luôn tận tình chỉ bảo, giúp đỡ và động viên em trong suốt quá trình nghiên cứu và hoàn thành đồ án chuyên ngành 2 này.

Mặc dù, đã cố gắng rất nhiều nhưng bài báo cáo không tránh khỏi những thiếu sót; em rất mong nhận được sự thông cảm, chỉ dẫn, giúp đỡ và đóng góp ý kiến của quý thầy cô, các cán bộ quản lý và các bạn đang học cùng em tại Trường Đại học Nam Cần Thơ.

Em xin chân thành cảm ơn!

Cần Thơ, ngày.....tháng.....năm 2026

Sinh viên thực hiện

Lê Quang Huy

LỜI CAM KẾT

Em xin cam kết đồ án chuyên ngành 2 này được hoàn thành dựa trên kết quả nghiên cứu của em và kết quả này chưa được sử dụng cho bất kỳ đồ án cơ sở nào khác.

Sinh viên thực hiện

Lê Quang Huy

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Cần Thơ, ngày ....tháng .....năm 2026

Giảng viên hướng dẫn

Trần Văn Thiện

MỤC LỤC

LỜI CẢM ƠN i
LỜI CAM KẾT ii
NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN iii
DANH MỤC BẢNG viii
DANH MỤC HÌNH ix
DANH MỤC TỪ VIẾT TẮT x
CHƯƠNG 1: GIỚI THIỆU 1
1.1 LÝ DO CHỌN ĐỀ TÀI 1
1.1.1.Gánh nặng từ các bệnh lý mãn tính trong xã hội hiện đại 1
1.1.2.Vai trò chiến lược của dinh dưỡng trong điều trị y khoa 1
1.1.3.Hạn chế của các hệ thống tư vấn dựa trên trí tuệ nhân tạo hiện nay 1
1.1.4.Sự cần thiết của giải pháp Đồ thị tri thức (Knowledge Graph) 2
1.2 SƠ LƯỢC VỀ CHƯƠNG TRÌNH 2
1.3 Ý NGHĨA CỦA ĐỀ TÀI 4
1.3.1 Ý nghĩa khoa học: 4
1.3.2 Ý nghĩa thực tiễn: 4
CHƯƠNG 2: CƠ SỞ LÝ LUẬN VÀ PHƯƠNG PHÁP NGHIÊN CỨU 5
2.1 PHÂN TÍCH YÊU CẦU 5
2.1.1 Xác định yêu cầu 5
2.1.2 Thu thập yêu cầu 6
2.1.3 Phân tích luồng xử lý của hệ thống 6
2.2 MÔ HÌNH HỆ THỐNG VÀ THIẾT KẾ 7
2.2.1 Mô hình hệ thống 7
2.2.2 Thiết kế đồ thị tri thức (Knowledge Graph Schema) 8
2.2.3 Thiết kế giao diện người dùng 10
2.3 PHÁT TRIỂN HỆ THỐNG 11
2.3.1 Lựa chọn công nghệ 11
2.3.2 Thu thập và Tiền Xử Lý Dữ Liệu Y Văn 12

2.3.2.1 Nguồn dữ liệu và Tiêu chí thu thập 12
2.3.2.2 Quy trình Tiền Xử Lý và Phân Đoạn (Chunking) 12
2.3.3 Trích Xuất Thực Thể và Quan Hệ bằng LLM 14
2.3.3.1 Cơ chế Few-Shot Prompting cho Open Information Extraction (OIE) 14
2.3.3.2 Quy trình 3 Pha của Framework EDC — Phân tích Chi Tiết 14
2.3.4 Xây Dựng và Chuẩn Hóa Đồ Thị Tri Thức 17
2.3.4.1 Hậu Xử Lý 2 Lớp 17
2.3.4.2 Import vào Neo4j và Chiến lược MERGE 18
2.4 TRIỂN KHAI VÀ DUY TRÌ 18
2.5 PHƯƠNG PHÁP NGHIÊN CỨU 19
CHƯƠNG 3: GIỚI THIỆU TỔNG QUAN VỀ VẤN ĐỀ NGHIÊN CỨU 21
3.1 TỔNG QUAN VỀ VẤN ĐỀ NGHIÊN CỨU 21
3.1.1 Khái niệm cơ bản 21
3.1.2 Mục tiêu của hệ thống Xây dựng đồ thị tri thức dinh dưỡng bệnh nhân 23
3.1.3 Phạm vi ứng dụng 24
3.1.4 Khả năng phát triển 24
3.1.5 Các yêu cầu hệ thống 25
3.2 KHẢO SÁT HIỆN TRẠNG 25
3.2.1 Các hệ thống tương tự hiện có 25
3.2.2 Phân tích khoảng trống và lý do cần giải pháp mới 26
CHƯƠNG 4: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG BẰNG NGÔN NGỮ UML. 28
4.1 MÔ HÌNH PHÂN RÃ CHỨC NĂNG (Business Function Diagram – BFD) 28
4.2 MÔ HÌNH QUAN HỆ THỰC THỂ - SCHEMA ĐỒ THỊ TRI THỨC (KG Schema) 28
4.3 MÔ HÌNH LUỒNG DỮ LIỆU (Data Flow Diagram – DFD) 29
4.3.1 Mức F0 Người Dùng/Admin 29
4.3.2 Mức F1 Người Dùng 29
4.3.3 Mức F1 Admin 30
4.4 MÔ HÌNH SƠ ĐỒ TUẦN TỰ (Sequence Diagram) 31
4.4.1 Luồng Tra cứu Dinh dưỡng (GraphRAG) 31
4.4.2 Pipeline Xây dựng KG (Admin/EDC) 31

4.5 MÔ HÌNH SƠ ĐỒ HOẠT ĐỘNG (Activity Diagram) 32
4.6 MÔ HÌNH KIẾN TRÚC THÀNH PHẦN HỆ THỐNG (Component Diagram). 33
4.7 MÔ HÌNH SƠ ĐỒ TRIỂN KHAI (Deployment Diagram) 34
CHƯƠNG 5: THIẾT KẾ CƠ SỞ DỮ LIỆU VÀ USE CASE 35
5.1 BẢNG CƠ SỞ DỮ LIỆU (Từ điển dữ liệu Entity) 35
5.2 BẢNG PHÁT HIỆN CÁC RÀNG BUỘC CƠ SỞ DỮ LIỆU (Constraints & Relationships) 36
5.2.1 Ràng buộc toàn vẹn (Graph Constraints) 36
5.2.2 Cấu trúc dữ liệu Quan hệ (Edge Schema) 36
5.3 SƠ ĐỒ USE CASE 38
CHƯƠNG 6: ĐẶC TẢ GIAO DIỆN 39
6.1 GIAO DIỆN TRANG CHỦ VÀ KHAI BÁO BỆNH LÝ (Home & Input Panel) 39
6.2 GIAO DIỆN NHẬN DIỆN MÓN ĂN AI (Vision Upload Panel) 41
6.3 GIAO DIỆN TƯ VẤN Y KHOA (GraphRAG Output) 41
6.4 GIAO DIỆN BIỂU ĐỒ VI CHẤT DINH DƯỠNG (Nutrient Metrics) 42
6.5 GIAO DIỆN QUẢN TRỊ CƠ SỞ DỮ LIỆU (Neo4j Browser Workspace) 43
6.6 GIAO DIỆN CẤU TRÚC TỔNG THỂ 43
6.7 GIAO DIỆN MẠNG LƯỚI CỦA MỘT CỤM THỰC PHẨM 44
6.8 GIAO DIỆN THUỘC TÍNH 16 VI CHẤT 44
6.9 GIAO DIỆN ĐẶC TẢ API (FastAPI Swagger UI) 45
CHƯƠNG 7: THỬ NGHIỆM VÀ ĐÁNH GIÁ 48
7.1 MÔI TRƯỜNG THỬ NGHIỆM 48
7.1.1 Cấu hình phần cứng 48
7.1.2 Cấu hình phần mềm 48
7.2 CÁC KỊCH BẢN THỬ NGHIỆM (Test Cases) 48
7.3 ĐÁNH GIÁ CHUNG 50
7.3.1 Tốc độ phản hồi và Hiệu năng (Latency) 50
7.3.2 Độ chính xác Nhận diện Ảnh (Vision AI Accuracy) 50
7.3.3 Tỷ lệ thành công Ánh xạ Ngữ nghĩa (Semantic Mapping Rate) 51
7.3.4 Điểm mạnh và hiệu quả (Ưu điểm) 51

7.3.5 Khó khăn / Điểm yếu tồn đọng 51
7.4 ĐÁNH GIÁ PIPELINE TRÍCH XUẤT ĐỒ THỊ TRI THỨC (BENCHMARK EDC) 52
7.4.1 Thiết lập Benchmark 52
7.4.2 Kết Quả Benchmark 52
7.4.3 Phân tích và Nhận xét 53
CHƯƠNG 8: KẾT LUẬN 55
8.1 ĐÁNH GIÁ TỔNG QUÁT ĐỒ ÁN 55
8.2 ĐIỂM MẠNH CỦA HỆ THỐNG 55
8.3 HẠN CHẾ CÒN TỒN ĐỌNG 55
8.4 ĐỊNH HƯỚNG PHÁT TRIỂN 56
TÀI LIỆU THAM KHẢO 57

DANH MỤC BẢNG

Bảng 2.1 Bảng yêu cầu chức năng.....5
Bảng 2.2 Bảng yêu cầu phi chức năng.....6
Bảng 2.3 Bảng lựa chọn công nghệ.....11
Bảng 2.4 Ví dụ minh họa kết quả Bước 3a — Top-3 ứng viên Schema.....16
Bảng 2.5 Kết quả ánh xạ ngữ nghĩa và xác nhận quan hệ tại Pha 3.....17
Bảng 3.1 Bảng yêu cầu phần cứng tối thiểu.....25
Bảng 3.2 Bảng yêu cầu phần mềm.....25
Bảng 3.3 Bảng Yêu cầu API Keys.....25
Bảng 3.4 Bảng so sánh tổng hợp các giải pháp hiện có.....26
Bảng 5.1 Bảng chi tiết thuộc tính thực thể Food (Thực phẩm).....35
Bảng 5.2 Bảng cấu trúc Nodes.....36
Bảng 5.3 Bảng cấu trúc dữ liệu quan hệ.....37
Bảng 7.1 Bảng cấu hình phần cứng.....48
Bảng 7.2 Bảng tốc độ phản hồi.....50
Bảng 7.3 Bảng độ chính xác nhận diện hình ảnh.....50
Bảng 7.4 Bảng tỉ lệ thành công ánh xạ ngữ nghĩa.....51
Bảng 7.5 Bảng cấu hình Benchmark.....52
Bảng 7.6 Điểm số tổng hợp (Total Scores) — 25 câu, 201 Triple đáp án chuẩn.....53
Bảng 7.7 Điểm số chi tiết theo từng thành phần Triple.....53
Bảng 7.8 Điểm số Triple hoàn chỉnh (Full Triple Score — khắt khe nhất).....53
Bảng 7.9 Bảng phân tích lỗi.....54

DANH MỤC HÌNH

Hình 4.1 Mô hình phân rã chức năng *28*
Hình 4.2 Mô hình quan hệ thực thể *28*
Hình 4.3 Mức F0 Người Dùng và Admin *29*
Hình 4.4 Mức F1 Người Dùng *29*
Hình 4.5 Mức F1 Admin *30*
Hình 4.6 Luồng Tra cứu Dinh dưỡng *31*
Hình 4.7 Pipeline Xây dựng KG *31*
Hình 4.8 Mô hình sơ đồ hoạt động *32*
Hình 4.9 Mô hình kiến trúc thành phần hệ thống *33*
Hình 4.10 Sơ đồ triển khai *34*
Hình 5.1 Sơ đồ Use Case *38*
Hình 6.1 Giao diện trang chủ *39*
Hình 6.2 Khai báo bệnh lý *40*
Hình 6.3 Nhận diện món ăn *41*
Hình 6.4 Giao diện tư vấn y khoa *42*
Hình 6.5 Biểu đồ vi chất *42*
Hình 6.6 Giao diện hệ quản trị CSDL Neo4j *43*
Hình 6.7 Giao diện cấu trúc tổng thể *43*
Hình 6.8 Giao diện mạng lưới của một cụm thực phẩm *44*
Hình 6.9 Giao diện thuộc tính 16 vi chất *44*
Hình 6.10 Giao diện tổng quan tài liệu kỹ thuật API (Swagger UI) của hệ thống. *45*
Hình 6.11 Chi tiết Endpoint kiểm tra trạng thái hoạt động của hệ thống (Read Root). *45*
Hình 6.12 Cấu trúc Endpoint xử lý truy vấn tư vấn dinh dưỡng (/api/chat). *46*
Hình 6.13 Cấu trúc Endpoint nhận diện thực phẩm qua hình ảnh (/api/vision). *47*
Hình 7.1 F1-Score theo từng thành phần *53*

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
AI	Artificial Intelligence
API	Application Programming Interface
CSDL	Cơ Sở Dữ Liệu
DB	Database
DFD	Data Flow Diagram
EDC	Extract-Define-Canonicalize
ERD	Entity-Relationship Diagram
GraphRAG	Graph Retrieval-Augmented Generation
KG	Knowledge Graph
LLM	Large Language Model
OIE	Open Information Extraction
RAG	Retrieval-Augmented Generation
Schema	Cấu trúc đồ thị
UI/UX	User Interface / User Experience
URI	Uniform Resource Identifier

CHƯƠNG 1: GIỚI THIỆU

1.1 LÝ DO CHỌN ĐỀ TÀI

1.1.1.Gánh nặng từ các bệnh lý mãn tính trong xã hội hiện đại

Trong những thập kỷ gần đây, cùng với sự thay đổi của lối sống và quá trình đô thị hóa, thế giới nói chung và Việt Nam nói riêng đang phải đối mặt với sự gia tăng nhanh chóng của các bệnh lý không lây nhiễm (NCDs).Các căn bệnh mãn tính như đái tháo đường (tiểu đường) [5], tăng huyết áp [8], suy thận [7], và các biến chứng tim mạch đã không còn là vấn đề của riêng cá nhân mà đã trở thành gánh nặng lớn cho hệ thống y tế công cộng và kinh tế xã hội.

Theo báo cáo của dự án Gánh nặng Bệnh tật Toàn cầu (GBD) do Viện Đo lường và Đánh giá Sức khỏe (IHME, Mỹ) chủ trì, tỷ lệ người mắc bệnh mãn tính đang có xu hướng trẻ hóa, gây ra những ảnh hưởng nghiêm trọng đến chất lượng cuộc sống và khả năng lao động.Đặc thù của các bệnh lý này là không thể điều trị dứt điểm trong thời gian ngắn mà đòi hỏi một quá trình quản lý, theo dõi và điều chỉnh thói quen sinh hoạt kéo dài suốt đời.

1.1.2.Vai trò chiến lược của dinh dưỡng trong điều trị y khoa

Đối với bệnh nhân mắc các bệnh lý mãn tính, việc dùng thuốc chỉ là một phần của phác đồ điều trị.Chế độ dinh dưỡng (Medical Nutrition Therapy - MNT) đóng vai trò then chốt, đôi khi mang tính quyết định đến tiến triển của bệnh.Một chế độ ăn uống khoa học, tuân thủ nghiêm ngặt các chỉ số về calo, đạm (protein), chất béo (lipid) và các vi chất không chỉ giúp kiểm soát chỉ số đường huyết hay huyết áp ở mức an toàn mà còn là phương pháp hiệu quả để ngăn ngừa các biến chứng nguy hiểm như suy tim, đột quỵ hay hoại tử chi.

Chế độ dinh dưỡng cho mỗi loại bệnh lý lại có những nguyên tắc đặc thù và thường trái chiều nhau.Chẳng hạn, người suy thận cần hạn chế protein và kali, trong khi người béo phì [6] lại cần ưu tiên protein chất lượng cao và cắt giảm tinh bột.Sự phức tạp này khiến cho thực đơn hằng ngày trở thành một một thách thức lớn đối với bệnh nhân và người nhà bệnh nhân.

1.1.3.Hạn chế của các hệ thống tư vấn dựa trên trí tuệ nhân tạo hiện nay

Sự ra đời của các mô hình ngôn ngữ lớn (LLM) như ChatGPT hay Llama đã mở ra kỷ nguyên mới cho các chatbot tư vấn.Tuy nhiên, khi áp dụng vào lĩnh vực nhạy cảm như y tế, các hệ thống này bộc lộ những hạn chế nghiêm trọng:

Hiện tượng ảo giác (AI Hallucination): LLM có xu hướng tạo ra các câu trả lời nghe rất thuyết phục nhưng thiếu căn cứ xác thực hoặc sai lệch về con số định lượng y khoa.

Thiếu khả năng suy luận logic trên dữ liệu cấu trúc: Các chatbot thông thường không thể truy xuất chính xác dữ liệu từ một bảng thành phần thực phẩm cố định mà thường "dự đoán" dựa trên xác suất từ ngữ.

Hệ thống hộp đen (Black-box): Người dùng không thể biết lời khuyên được đưa ra dựa trên tài liệu hay quy định y tế nào, dẫn đến sự thiếu tin tưởng từ phía bệnh nhân và bác sĩ.

1.1.4.Sự cần thiết của giải pháp Đồ thị tri thức (Knowledge Graph)

Xuất phát từ những khoảng trống đó, đề tài "Xây dựng Đồ thị Tri thức Dinh dưỡng Bệnh nhân" được đề xuất và triển khai.Đề tài ứng dụng công nghệ Đồ thị Tri thức (Knowledge Graph) kết hợp với kiến trúc GraphRAG (Graph-based Retrieval Augmented Generation) nhằm tạo ra một hệ thống tư vấn dinh dưỡng y khoa có khả năng:

Hệ thống tự động hóa quá trình trích xuất tri thức từ tài liệu y khoa tiếng Việt dạng thô thông qua framework EDC (Extract – Define – Canonicalize) [1].

Hệ thống lưu trữ tri thức dưới dạng đồ thị cấu trúc (Thực phẩm – Vi chất – Bệnh lý) trong cơ sở dữ liệu Neo4j [2], giúp kết nối các khái niệm y khoa một cách trực quan và có thể truy vấn.

Hệ thống tích hợp LLM (Llama 3.3) với đồ thị tri thức để sinh ra lời khuyên dinh dưỡng cá nhân hóa, có kiểm chứng và giảm thiểu tối đa ảo giác.

1.2 SƠ LƯỢC VỀ CHƯƠNG TRÌNH

Hệ thống được xây dựng gồm hai pipeline chính phối hợp chặt chẽ với nhau:

Pipeline 1 – Xây dựng Knowledge Graph (Offline):

Đây là giai đoạn tự động hóa toàn bộ quá trình chuyển đổi văn bản y khoa thô thành đồ thị tri thức có cấu trúc.Pipeline này áp dụng framework EDC (Extract – Define – Canonicalize) [1] gồm ba bước nối tiếp:

(1) Extract (Trích xuất): Sử dụng mô hình Llama-3.3-70b-versatile kết hợp với cơ chế few-shot prompting để thực hiện Open Information Extraction (OIE) — tự động chiết xuất các bộ ba thực thể (Subject, Relation, Object) từ tài liệu y khoa.Ví dụ: từ câu *"Muối natri làm tăng huyết áp" [8]* hệ thống trích ra Triple [muối natri, là yếu tố nguy cơ của, tăng huyết áp].

(2) Define (Định nghĩa lược đồ): Tự động định nghĩa tập hợp các loại quan hệ (Relation Schema) từ Triple thô, đảm bảo tính nhất quán về ngữ nghĩa trong toàn bộ đồ thị.

(3) Canonicalize (Chuẩn hóa): Ánh xạ các quan hệ đa dạng về cùng một nhãn quan hệ chuẩn (canonical form) bằng Embedding Similarity, giải quyết vấn đề từ đồng nghĩa như *"cần hạn chế ở"* và *"làm trầm trọng"* đều cùng chỉ mối quan hệ "có hại".

Sau bước trích xuất, toàn bộ Triple thô được qua **hậu xử lý 2 lớp**:

Lớp 1 – Rule-based Cleaning: Lọc bỏ các Triple trừu tượng, chủ thể chung chung và sửa chiều quan hệ bị đảo ngược.

Lớp 2 – Semantic Deduplication bằng Jina Embeddings v3: Gom nhóm các thực thể đồng nghĩa bằng Cosine Similarity (ngưỡng 0.90) để khử trùng lặp ngữ nghĩa.

Cuối cùng, Triple sạch được nạp vào Neo4j Graph Database [2] theo phân loại 4 nhãn node: Food, Disease, Nutrient, Other.Đồng thời, 16 thuộc tính định lượng dinh dưỡng (Calo, Protein, Chất béo, Carbs, Canxi, Sắt, Vitamin A/B1/C...) được bổ sung trực tiếp vào node Thực phẩm từ bộ dữ liệu Excel.

Pipeline 2 – Tư vấn GraphRAG (Online/Real-time):

Khi người dùng gửi yêu cầu tư vấn (nhập tên món ăn hoặc chụp ảnh), hệ thống thực hiện tuần tự:

1. **Nhận diện ảnh (Vision AI):** Mô hình llama-4-scout-17b-16e-instruct tự động nhận diện tên món ăn từ ảnh tải lên.
2. **Truy vấn Neo4j:** Tìm kiếm node thực phẩm và danh sách dưỡng chất cần tránh theo bệnh lý bằng Cypher Query.
3. **Semantic Fallback:** Nếu không tìm thấy chính xác, dùng Llama 3.3 để ánh xạ ngữ nghĩa từ input sang node gần nhất trong KG.
4. **Circuit Breaker:** Nếu sau cả 2 bước tìm kiếm vẫn không có dữ liệu, hệ thống từ chối sinh lời khuyên và hiển thị thông báo rõ ràng — đảm bảo ít ảo giác.
5. **Sinh lời khuyên:** Llama 3.3 tổng hợp dữ liệu từ đồ thị thành lời tư vấn y khoa bằng tiếng Việt, chỉ dựa trên ngữ cảnh tri thức đã xác minh.

Hạ tầng triển khai gồm 4 micro-services đóng gói qua Docker Compose: Neo4j Graph DB [2], FastAPI Backend, React/Vite Frontend và Nginx API Gateway.

1.3 Ý NGHĨA CỦA ĐỀ TÀI

1.3.1 Ý nghĩa khoa học:

Đề tài đóng góp vào lĩnh vực giao thoa giữa Khai phá Tri thức (Knowledge Extraction), Đồ thị Tri thức (Knowledge Graph) và Sinh từ Tăng cường Truy xuất (RAG) trong miền y tế tiếng Việt — một lĩnh vực còn rất ít nghiên cứu ứng dụng thực tế trong nước.Cụ thể:

Chứng minh khả năng áp dụng framework EDC [1] cho ngôn ngữ và miền kiến thức chuyên ngành tiếng Việt (y khoa dinh dưỡng), đặt nền móng cho các nghiên cứu mở rộng sang các chuyên ngành khác.

Đề xuất quy trình hậu xử lý Knowledge Graph 2 tầng (Rule-based + Semantic Embedding) phù hợp với đặc thù dữ liệu tiếng Việt — nơi từ đồng nghĩa và cách diễn đạt biến thể rất phong phú.

Nghiên cứu và giải quyết vấn đề ảo giác LLM trong hệ thống tư vấn y tế thông qua cơ chế Circuit Breaker — chỉ sinh lời khuyên khi có cơ sở tri thức đã kiểm chứng.

1.3.2 Ý nghĩa thực tiễn:

Hỗ trợ người bệnh tra cứu trực quan: Thay vì lật từng trang cẩm nang, người bệnh có thể nhập tên món ăn hoặc chụp ảnh để nhận ngay lời khuyên dinh dưỡng cá nhân hóa, dễ hiểu và đáng tin cậy.

Tiết kiệm thời gian và công sức số hóa tri thức: Pipeline KG Builder tự động hóa quá trình trích xuất tri thức y khoa vốn đòi hỏi nhiều công sức thủ công từ chuyên gia.

Nền tảng mở rộng linh hoạt: Hệ thống được thiết kế mô-đun, cho phép bổ sung bệnh lý mới hoặc tập dữ liệu thực phẩm mới chỉ bằng cách chạy lại pipeline import — không cần sửa đổi code.

Đặt nền móng cho ứng dụng y tế AI có trách nhiệm: Kiến trúc GraphRAG với cơ chế chống ảo giác là mô hình tham chiếu quan trọng cho các hệ thống AI y tế cần độ tin cậy cao trong bối cảnh Việt Nam.

CHƯƠNG 2: CƠ SỞ LÝ LUẬN VÀ PHƯƠNG PHÁP NGHIÊN CỨU

2.1 PHÂN TÍCH YÊU CẦU

2.1.1 Xác định yêu cầu

Hệ thống "Xây dựng Đồ thị Tri thức Dinh dưỡng Bệnh nhân" cần đáp ứng hai nhóm yêu cầu cốt lõi:

Yêu cầu chức năng:

Mã	Yêu cầu	Mức ưu tiên
F01	Trích xuất tự động Triple tri thức (Subject, Relation, Object) từ văn bản y khoa tiếng Việt	Cao
F02	Làm sạch, chuẩn hóa và khử trùng lặp ngữ nghĩa các Triple sau trích xuất	Cao
F03	Lưu trữ đồ thị tri thức (Food, Disease, Nutrient) vào Neo4j Graph Database	Cao
F04	Tra cứu thông tin dinh dưỡng và quan hệ bệnh lý của một thực phẩm	Cao
F05	Sinh lời khuyên dinh dưỡng bằng tiếng Việt thông qua LLM dựa trên ngữ cảnh từ KG	Cao
F06	Nhận diện tên món ăn từ ảnh người dùng tải lên (Vision AI)	Trung bình
F07	Hiển thị trực quan biểu đồ thành phần dinh dưỡng dựa trên dữ liệu chuẩn [9]	Trung bình
F08	Cảnh báo rõ ràng khi thực phẩm chưa có trong cơ sở dữ liệu hệ thống	Cao
F09	Hỗ trợ tư vấn theo bốn nhóm bệnh: Tiểu đường, Tăng huyết áp, Suy thận, Béo phì	Cao

Bảng 2.1 Bảng yêu cầu chức năng

Yêu cầu phi chức năng:

Mã	Yêu cầu	Mức ưu tiên
NF01	Thời gian phản hồi lời khuyên AI ≤ 10 giây trong điều kiện bình thường	Cao
NF02	Hệ thống đảm bảo ít ảo giác — chỉ tư vấn khi có dữ liệu xác minh trong KG	Cao
NF03	Giao diện người dùng thân thiện, hoạt động ổn định trên trình duyệt web hiện đại	Trung bình
NF04	Toàn bộ dịch vụ vận hành nhất quán trong môi trường Docker Compose	Cao
NF05	Dữ liệu API Key và thông tin nhạy cảm được quản lý qua biến môi trường (.env)	Cao

Bảng 2.2 Bảng yêu cầu phi chức năng

2.1.2 Thu thập yêu cầu

Quá trình thu thập yêu cầu cho hệ thống được thực hiện qua 3 nguồn chính:

a.Phân tích tài liệu y khoa: Nghiên cứu các hướng dẫn dinh dưỡng lâm sàng của Bộ Y tế Việt Nam và Viện Dinh dưỡng Quốc gia cho bốn nhóm bệnh lý mục tiêu.Từ đó xác định các mối quan hệ dinh dưỡng quan trọng cần mô hình hóa trong đồ thị: thực phẩm nào là "yếu tố nguy cơ", loại vi chất nào "cần hạn chế" hay "cần bổ sung" theo từng bệnh.

b.Khảo sát bộ dữ liệu thực phẩm Việt Nam: Thu thập và chuẩn hóa bảng thành phần thực phẩm Việt Nam gồm hơn 150 loại thực phẩm phổ biến trong bữa ăn hàng ngày, với đầy đủ 16 chỉ số vi chất định lượng (Calo, Protein, Chất béo, Carbs, Canxi, Sắt, Kali, Natri, Vitamin A/B1/C, Beta Caroten...).

c.Thực nghiệm với framework EDC [1]: Chạy thử pipeline trích xuất trên tập dữ liệu mẫu để xác định các loại nhiễu phổ biến (Triple trừu tượng, quan hệ đảo chiều, thực thể đồng nghĩa) — từ đó thiết kế bộ quy tắc lọc và ngưỡng Embedding Similarity phù hợp.

2.1.3 Phân tích luồng xử lý của hệ thống

Hệ thống hoạt động theo hai luồng độc lập nhưng liên kết chặt chẽ:

Luồng 1 – Xây dựng tri thức (Knowledge Building Flow):

Tài liệu Y khoa (.txt)

Chunking → Phân đoạn văn bản thành các đoạn có kích thước phù hợp

OIE (LLM Prompting) → Trích xuất Triple thô (Subject, Relation, Object)

Schema Definition → Xây dựng danh sách các loại quan hệ

Schema Canonicalization (Embedding) → Chuẩn hóa Triple theo schema

Rule-based Cleaning → Loại bỏ các Triple trừu tượng hoặc đảo chiều

Semantic Deduplication (Jina API) → Gom nhóm các thực thể đồng nghĩa

Neo4j Import → Xây dựng đồ thị tri thức hoàn chỉnh và xử lý các thực thể đồng nghĩa

Luồng 2 – Tư vấn thời gian thực (GraphRAG Query Flow):

Người dùng (Tên món / Ảnh + Bệnh lý)

Vision AI → Nhận diện và xác định tên món ăn từ ảnh (nếu có)

Neo4j Exact Lookup → Truy vấn chính xác dữ liệu dinh dưỡng và danh sách kiêng kỵ

Semantic Mapping (LLM) → Ánh xạ ngữ nghĩa khi không tìm thấy kết quả trùng khớp

Circuit Breaker → Kích hoạt cảnh báo khi dữ liệu không khả dụng

LLM Response Generation → Tạo lời khuyên y khoa bằng tiếng Việt

Người dùng → Nhận kết quả gồm lời khuyên và biểu đồ thành phần dinh dưỡng [9].

2.2 MÔ HÌNH HỆ THỐNG VÀ THIẾT KẾ

2.2.1 Mô hình hệ thống

Hệ thống được xây dựng theo kiến trúc Microservices gồm 4 thành phần độc lập, giao tiếp qua mạng nội bộ Docker:

Nginx Gateway: Tiếp nhận toàn bộ request từ người dùng và phân luồng tới Frontend (React) hoặc Backend (FastAPI) dựa trên đường dẫn URL.

React Frontend: Giao diện người dùng (SPA) sử dụng Vite làm build tool, tích hợp biểu đồ dinh dưỡng, upload ảnh và hiển thị lời khuyên AI theo thời gian thực.

FastAPI Backend: Nhân xử lý nghiệp vụ — điều phối luồng GraphRAG, gọi Groq API, truy vấn Neo4j và trả về kết quả dạng JSON.

Neo4j Graph DB: Kho lưu trữ trung tâm của đồ thị tri thức dinh dưỡng, cung cấp giao diện Cypher Query và REST API.

2.2.2 Thiết kế đồ thị tri thức (Knowledge Graph Schema)

Khác với cơ sở dữ liệu quan hệ truyền thống, đồ thị tri thức được thiết kế theo mô hình Property Graph với 4 nhãn node và 12 nhóm quan hệ:

Node Labels (Nhãn thực thể):

Thực thể: Food (Thực phẩm):

Mô tả: Thực phẩm Việt Nam

Các thuộc tính: name, calories, protein, fat, carbs, fiber, calcium, iron, sodium, potassium, vitamin_a, vitamin_b1, vitamin_c, beta_carotene, cholesterol, phosphorus.

Thực thể Disease (Bệnh lý):

Mô tả: Bệnh lý mãn tính (4 nhóm)

Các thuộc tính: name.

Thực thể Nutrient (Dưỡng chất):

Mô tả: Vi chất dinh dưỡng.

Các thuộc tính: name.

Thực thể Other (Thực thể khác):

Mô tả: Thực thể khác trong văn bản.

Các thuộc tính: name.

Relationship Types (Quan hệ):

Relationship Types (Quan hệ):

- **Quan hệ: làm trầm trọng**
  - Chiều: Food / Nutrient → Disease
  - Ý nghĩa: Thực phẩm/dưỡng chất làm xấu đi triệu chứng hoặc thúc đẩy tiến triển bệnh.
- **Quan hệ: được khuyến nghị cho**

    - Chiều: Food / Nutrient → Disease
    - Ý nghĩa: Được chuyên gia y tế khuyến nghị cho bệnh nhân mắc bệnh này.
- **Quan hệ: cần hạn chế ở**
    - Chiều: Food / Nutrient → Disease
    - Ý nghĩa: Cần giảm lượng tiêu thụ ở bệnh nhân mắc bệnh.
- **Quan hệ: chống chỉ định với**
    - Chiều: Food → Disease
    - Ý nghĩa: Gây hại và cần tránh hoàn toàn ở bệnh nhân.
- **Quan hệ: phòng ngừa**
    - Chiều: Food / Nutrient → Disease
    - Ý nghĩa: Giúp ngăn ngừa bệnh hoặc biến chứng khi tiêu thụ đầy đủ.
- **Quan hệ: hỗ trợ**
    - Chiều: Food / Nutrient → Disease
    - Ý nghĩa: Hỗ trợ kiểm soát hoặc cải thiện tình trạng bệnh.
- **Quan hệ: ảnh hưởng đường huyết**
    - Chiều: Food → Disease / Other
    - Ý nghĩa: Ảnh hưởng mức đường huyết theo chiều hướng: tăng / tăng nhẹ / giảm.
- **Quan hệ: giàu**
    - Chiều: Food → Nutrient
    - Ý nghĩa: Thực phẩm là nguồn giàu dưỡng chất, hàm lượng cao hơn mức trung bình.
- **Quan hệ: ít**
    - Chiều: Food → Nutrient
    - Ý nghĩa: Thực phẩm có hàm lượng thấp của chỉ số dinh dưỡng.
- **Quan hệ: thiếu hụt gây ra**
    - Chiều: Nutrient → Disease

    - Ý nghĩa: Thiếu hụt dưỡng chất dẫn đến bệnh hoặc tình trạng sức khỏe.
- **Quan hệ: tương tác với**
    - Chiều: Nutrient → Nutrient / Other
    - Ý nghĩa: Tương tác — tăng cường hoặc cản trở hấp thu/chuyển hóa.
- **Quan hệ: là yếu tố nguy cơ của**
    - Chiều: Food / Nutrient → Disease
    - Ý nghĩa: Làm tăng đáng kể nguy cơ mắc bệnh.

Ví dụ Triple trong đồ thị:

(muối) -[là yếu tố nguy cơ của]-> (tăng huyết áp)

(thịt đỏ) -[làm trầm trọng]-> (suy thận)

(rau xanh) -[tốt cho]-> (béo phì)

(gạo lứt) -[chứa]-> (chất xơ)

2.2.3 Thiết kế giao diện người dùng

Giao diện được thiết kế theo phong cách Medical Theme với các thành phần chính:

Bảng điều khiển tư vấn: Panel trung tâm gồm: (1) Dropdown chọn bệnh lý; (2) Ô nhập tên món ăn; (3) Nút upload ảnh với preview; (4) Nút gửi truy vấn.

Bảng kết quả dinh dưỡng chi tiết: Hiển thị đầy đủ 16 chỉ số vi chất dạng card layout, kèm biểu đồ cột so sánh trực quan.

Panel Phân tích y khoa AI: Hiển thị lời khuyên từ Llama 3.3 [3] theo định dạng Markdown có cấu trúc (heading, bullet, in đậm), phân biệt rõ phần "Lời khuyên" và "Gợi ý thay thế" (nếu có).

Thông báo cảnh báo: Banner màu vàng nổi bật hiển thị khi thực phẩm không có trong cơ sở dữ liệu hệ thống.

2.3 PHÁT TRIỂN HỆ THỐNG

2.3.1 Lựa chọn công nghệ

Thành phần	Công nghệ	Phiên bản	Lý do lựa chọn
Graph Database	Neo4j	5.16	Hệ CSDL đồ thị phổ biến nhất, hỗ trợ Cypher Query mạnh mẽ, có giao diện visual browser
KG Extraction LLM	Llama 3.3 (via Groq) [3]	70b-versatile	Mô hình mã nguồn mở hàng đầu, tích hợp API tốc độ cao qua Groq, hỗ trợ tiếng Việt
Vision LLM	Llama 4 Scout (via Groq)	17b-16e-instruct	Mô hình đa phương thức, nhận diện ảnh thực phẩm chính xác
Semantic Embedding	Jina Embeddings v3	Cloud API	Vector hóa tiếng Việt chất lượng cao, không cần GPU nội bộ
Backend Framework	FastAPI + Python	3.11	Async REST API, auto-docs, tích hợp tốt với neo4j-driver
Frontend Framework	React + Vite	React 18	SPA hiện đại, Hot Module Replacement, build nhanh
Reverse Proxy	Nginx	1.29	Gateway nhẹ, cấu hình đơn giản, xử lý proxy_pass hiệu quả
Container	Docker Compose	v3	Đóng gói nhất quán, dễ triển khai đa môi trường

Bảng 2.3 Bảng lựa chọn công nghệ

Chi tiết pipeline EDC Framework:

Framework EDC (Extract-Define-Canonicalize) [1] là bộ khung trích xuất tri thức từ văn bản tự do được ứng dụng và tùy chỉnh trong đề tài này.Ba module chính được tích hợp:

edc/extract.py – OIE Extractor: Nhận văn bản đầu vào và few-shot examples, gọi LLM API để trích xuất danh sách Triple (Subject, Relation, Object).

edc/schema_definition.py – SchemaDefiner: Phân tích tập Triple để tự động đề xuất tập hợp các nhãn quan hệ đặc thù cho miền dinh dưỡng/bệnh lý.

edc/schema_canonicalization.py – SchemaCanonicalizer: Dùng Embedding Similarity thông qua **Jina Embeddings v3** để ánh xạ quan hệ thô về lược đồ chuẩn, giải quyết vấn đề đa dạng ngôn ngữ trong văn bản y khoa tiếng Việt.

2.3.2 Thu thập và Tiền Xử Lý Dữ Liệu Y Văn

2.3.2.1 Nguồn dữ liệu và Tiêu chí thu thập

Dữ liệu đầu vào cho pipeline xây dựng Knowledge Graph gồm hai nguồn chính, phục vụ hai mục đích khác nhau nhưng bổ sung lẫn nhau:

Nguồn 1 – Tài liệu y khoa lâm sàng (dạng văn bản phi cấu trúc): Các văn bản hướng dẫn điều trị và dinh dưỡng lâm sàng cho 4 nhóm bệnh mãn tính (Tiểu đường, Tăng huyết áp, Suy thận, Béo phì) [5][7][8][6], được thu thập từ Viện Dinh Dưỡng Quốc gia và Bộ Y tế Việt Nam với tổng dung lượng khoảng 40.000 từ. Văn bản được chọn lọc theo tiêu chí: Có độ tin cậy khoa học cao (nguồn chính thức), Chứa nhiều mối quan hệ nhân quả giữa thực phẩm và bệnh lý, Không trùng lặp nội dung giữa các nguồn.

Nguồn 2 – Bảng thành phần thực phẩm Việt Nam (dữ liệu có cấu trúc): Bộ dữ liệu định lượng 16 vi chất dinh dưỡng cho 150+ loại thực phẩm phổ biến, được nhập từ file Excel theo Bảng Thành Phần Thực Phẩm Việt Nam do Viện Dinh Dưỡng Quốc gia ban hành [9]. Đây là nguồn dữ liệu số học bổ sung trực tiếp vào node Food trong Neo4j.

2.3.2.2 Quy trình Tiền Xử Lý và Phân Đoạn (Chunking)

Do giới hạn cửa sổ ngữ cảnh (Context Window) của các API LLM, toàn bộ tài liệu y khoa không thể đưa vào một lần gọi API mà cần được phân đoạn thành các đoạn văn nhỏ hơn (chunks). Đây là bước tiền xử lý bắt buộc và có ảnh hưởng trực tiếp đến chất lượng Triple trích xuất. Để đưa văn bản thô về dạng phù hợp nhất cho trích xuất tri thức, đề tài triển khai quy trình tiền xử lý hai bước độc lập theo thứ tự:

Bước 1 – Chuẩn hóa sang văn xuôi

Bước đầu tiên giải quyết vấn đề cấu trúc không đồng nhất. Thay vì viết lại các quy tắc chuyển đổi cứng nhắc bằng code, đề tài sử dụng LLM (Llama 3.3-70B) với nhiệt độ sinh chữ thấp (temperature = 0.1) như một công cụ chuẩn hóa cấu trúc thông minh. LLM nhận đầu vào là toàn bộ văn bản thô và được hướng dẫn bởi 10 quy tắc vàng (Strict Rules) để đảm bảo đầu ra là văn xuôi tự nhiên, hoàn chỉnh và không mất mát thông tin:

Bảo toàn dữ liệu định lượng: Tuyệt đối giữ toàn bộ con số, đơn vị (mg, %, ml/kg, g/ngày) và khoảng giới hạn dinh dưỡng. Đây là dữ liệu không thể suy ra lại nếu bị mất.

Loại bỏ nhiễu (Noise Removal): Xóa toàn bộ tên tác giả, chức danh, header bài báo khoa học, số trang, link URL, mục Tài liệu tham khảo.

Chống lặp từ và bảo đảm tính hoàn chỉnh: LLM không được tóm tắt hay lược bỏ bất kỳ đoạn y khoa nào, đặc biệt là phần thực đơn mẫu, lượng nước và định lượng cụ thể.

Kết quả bước 1: Từ file *_raw.txt có bảng biểu và siêu dữ liệu → file *.txt dạng văn xuôi tiếng Việt liền mạch, sẵn sàng cho bước chunking.

Bước 2 – Phân Đoạn và Giải Quyết Đồng Tham Chiếu (file preprocess_document.py)

Bước thứ hai xử lý văn bản đã được chuẩn hóa ở bước 1 qua 3 giai đoạn nhỏ:

Giai đoạn 2a – Làm sạch (Clean): Hàm clean_raw_text() loại bỏ các đoạn trùng lặp, khoảng trắng thừa và các mẫu văn bản đặc thù (header, số trang) bằng regex.

Giai đoạn 2b – *Tách câu và Phân đoạn (Chunking):* Hàm split_into_sentences() tách văn bản thành danh sách câu bằng dấu câu kết thúc (., !, ?). Để tránh tách nhầm tại các số thập phân (0.8 g/kg) hoặc viết tắt đơn vị (mg., kg.), hệ thống bảo vệ tạm thời các chuỗi này bằng placeholder <DOT> trước khi tách, rồi khôi phục lại sau. Các câu sau đó được gom thành chunk theo đơn vị 3 câu/chunk bởi hàm chunk_sentences().

Giai đoạn 2c – *Giải Quyết Đồng Tham Chiếu (Coreference Resolution):* Đây là điểm kỹ thuật nổi bật nhất của bước tiền xử lý. Khi chia văn bản thành các chunk độc lập, một vấn đề nghiêm trọng phát sinh: các đại từ và cụm tham chiếu ("nó", "điều này", "phương pháp này", "các khuyến nghị đó") trong một chunk có thể mang nghĩa từ chunk trước đó. Ví dụ:

Chunk 1: *"Gạo lứt chứa nhiều chất xơ và giúp kiểm soát đường huyết."*
Chunk 2: *"Điều này đặc biệt có lợi cho bệnh nhân tiểu đường."*

Nếu Extract Chunk 2 mà không có ngữ cảnh, LLM sẽ không biết "Điều này" là gì và sẽ tạo ra Triple vô nghĩa. Để giải quyết, hệ thống gọi LLM (Llama 3.3-70B với temperature = 0.0) cho từng chunk, cung cấp đoạn chunk liền trước làm ngữ cảnh (Sliding Window kích thước 1). LLM có nhiệm vụ thay thế tất cả đại từ và cụm tham chiếu mơ hồ bằng tên thực thể cụ thể, trả về đoạn văn đã được làm rõ ràng. Ví dụ:

Chunk 2 sau Coreference Resolution: *"Việc gạo lứt chứa nhiều chất xơ đặc biệt có lợi cho bệnh nhân tiểu đường."*

Kết quả cuối cùng của toàn bộ pipeline tiền xử lý hai bước: Mỗi file bệnh lý được xuất ra dạng file .txt trong đó mỗi dòng là một chunk 3 câu, tự chứa đầy đủ ngữ nghĩa, sẵn sàng đưa vào module OIE của EDC mà không cần thêm bất kỳ bước xử lý nào.

2.3.3 Trích Xuất Thực Thể và Quan Hệ bằng LLM

2.3.3.1 Cơ chế Few-Shot Prompting cho Open Information Extraction (OIE)

Thay vì sử dụng các mô hình nhận dạng thực thể (NER) truyền thống đòi hỏi dữ liệu gán nhãn thủ công, đề tài áp dụng kỹ thuật Few-Shot Prompting — cung cấp cho LLM một số ví dụ mẫu ngay trong câu lệnh để LLM tự học cách trích xuất đúng định dạng, mà không cần huấn luyện lại mô hình. Cấu trúc một few-shot example trong file oie_few_shot_examples.txt:

Input: "Chế độ ăn nhiều muối làm tăng huyết áp và gia tăng nguy cơ biến chứng tim mạch."

Output:

- (muối, là yếu tố nguy cơ của, tăng huyết áp)
- (muối, là yếu tố nguy cơ của, biến chứng tim mạch)

Mỗi few-shot example được chọn lọc kỹ để bao phủ đầy đủ các loại quan hệ y khoa: tích cực (tốt cho, phòng ngừa), tiêu cực (làm trầm trọng, chống chỉ định), và trung lập (chứa, cần hạn chế ở). Tổng cộng 10 ví dụ mẫu được chuẩn bị cho module OIE của bệnh lý, và 8 ví dụ cho module dinh dưỡng thực phẩm.

2.3.3.2 Quy trình 3 Pha của Framework EDC — Phân tích Chi Tiết

Framework EDC xử lý từng chunk văn bản một cách tuần tự qua 3 pha nối tiếp nhau, sau đó tổng hợp kết quả toàn bộ tài liệu vào cuối. Kiến trúc này đảm bảo mỗi Triple được đi qua một quy trình kiểm soát chất lượng chặt chẽ trước khi được công nhận là hợp lệ.

Pha 1 — EXTRACT: Trích xuất Triple thô (Open Information Extraction)

Ở pha đầu tiên, hệ thống áp dụng kỹ thuật Trích xuất Thông tin Mở (Open Information Extraction – OIE) thông qua mô hình Llama 3.3-70B. Điểm đặc biệt của phương pháp này là Relation (quan hệ giữa hai thực thể) hoàn toàn không bị ràng buộc bởi bất kỳ danh mục định sẵn nào — LLM được tự do diễn đạt quan hệ bằng bất kỳ cụm từ nào phản ánh đúng nghĩa của câu gốc.

Để định hướng mô hình cho đặc thù miền y khoa dinh dưỡng, hệ thống cung cấp cho LLM 5 ví dụ mẫu được thiết kế thủ công (few-shot examples), mỗi

ví dụ tương ứng một nhóm bệnh lý khác nhau (Tiểu đường, Tăng huyết áp, Béo phì, Suy thận). Các ví dụ này bao phủ đầy đủ hai chiều quan hệ y khoa: tích cực (tốt cho, phòng ngừa) và tiêu cực (làm trầm trọng, chống chỉ định). Ví dụ:

Đầu vào: "Natri làm co mạch máu, làm trầm trọng cao huyết áp và cần hạn chế dưới 2300mg/ngày. Rượu bia chống chỉ định với cao huyết áp."

Đầu ra:

(natri, làm trầm trọng, cao huyết áp)

(natri, cần hạn chế ở, cao huyết áp)

(rượu bia, chống chỉ định với, cao huyết áp)

Một yêu cầu quan trọng được nhúng trong các ví dụ mẫu: tên thực thể phải là khái niệm sạch, không kèm số liệu hay đơn vị đo (ví dụ: phải là "natri", không phải "natri 2300mg"). Điều này đảm bảo các thực thể có thể được gom nhóm và trùng lặp chính xác ở bước hậu xử lý sau.

Kết quả Pha 1 là một tập Triple thô (Subject, Relation, Object) trong đó Relation là chuỗi tự do, gần như mỗi chunk sẽ sản sinh ra một Relation với cách diễn đạt khác nhau dù ý nghĩa có thể tương đồng

Pha 2 — DEFINE: Định nghĩa ngữ nghĩa Schema Quan hệ (Schema Definition)

Pha 2 giải quyết bài toán cốt lõi: từ tập Relation tự do đa dạng ở Pha 1, làm thế nào để so sánh ngữ nghĩa của chúng với Schema chuẩn một cách chính xác?

Câu trả lời là xây dựng một "cầu nối ngữ nghĩa" — thay vì so sánh trực tiếp chuỗi ký tự (vốn rất dễ thất bại do khác biệt từ ngữ), hệ thống yêu cầu LLM viết một đoạn định nghĩa ngữ nghĩa cho mỗi Relation thô dựa trên ngữ cảnh của đoạn văn và các Triple vừa trích xuất.

Trước tiên, toàn bộ Relation thô được lọc trùng lặp — nếu Pha 1 cho ra 8 Triple nhưng chỉ có 4 loại Relation khác nhau, Pha 2 chỉ cần viết định nghĩa cho 4 loại đó. Đây là bước tối ưu hóa tiết kiệm tài nguyên API đáng kể.

Kết quả Pha 2 là một từ điển ánh xạ: Relation thô → Định nghĩa ngữ nghĩa đầy đủ. Ví dụ:

"làm co mạch khiến tăng huyết áp"

→ "Thực thể nguồn có tác động sinh lý trực tiếp lên hệ tuần hoàn, gây ra tình trạng co thắt mạch máu dẫn đến tăng áp lực lên thành mạch, làm trầm trọng thêm bệnh tăng huyết áp."

Chính các định nghĩa này — chứ không phải chuỗi Relation thô — sẽ được sử dụng để tính toán độ tương đồng ngữ nghĩa với Schema ở Pha 3. Đây là lý do tại sao EDC vượt trội hơn so với các phương pháp keyword matching hoặc exact string matching thông thường.

Pha 3 — CANONICALIZE: Chuẩn hóa về Schema chính thức (Schema Canonicalization)

Đây là pha kỹ thuật phức tạp nhất và đồng thời là đóng góp cốt lõi nhất của toàn bộ framework. Pha 3 thực hiện hai bước con nối tiếp cho từng Triple:

Bước 3a — Tìm kiếm ứng viên bằng Embedding Similarity:

Định nghĩa ngữ nghĩa của Relation thô (từ Pha 2) được chuyển thành vector số học (embedding) bởi Jina Embeddings v3. Đồng thời, định nghĩa của 12 nhãn quan hệ chuẩn trong Schema đã được vector hóa từ trước và lưu sẵn để tái sử dụng. Hệ thống tính Cosine Similarity giữa vector của Relation thô và 12 vector Schema, sau đó chọn ra top-5 nhãn Schema có điểm tương đồng cao nhất làm danh sách ứng viên.

Bước này hoạt động hoàn toàn trên phép tính toán nội bộ (vector multiplication), không gọi thêm API nào — đảm bảo tốc độ xử lý cao.

Bước 3b — Xác nhận lựa chọn bằng LLM (Verification):

Với danh sách top-5 ứng viên, hệ thống trình bày cho LLM một bài kiểm tra trắc nghiệm nhiều lựa chọn (multiple-choice): đoạn văn gốc, Triple cần chuẩn hóa và 5 lựa chọn nhãn Schema kèm định nghĩa tương ứng, cùng lựa chọn F là "Không khớp nhãn nào". LLM trả về đúng một ký tự duy nhất (A/B/C/D/E/F). Thiết kế này tối giản hóa token tiêu thụ (chỉ cần 1 token đầu ra) trong khi vẫn tận dụng khả năng suy luận ngữ nghĩa sâu của LLM.

Quyết định cuối của Pha 3 cho từng Triple:

Trường hợp	**Kết quả**
LLM chọn một trong A–E	Triple được ánh xạ về nhãn Schema tương ứng → lưu vào KG
LLM chọn F (None of the above)	Triple bị loại bỏ khỏi tập kết quả
Relation thô đã khớp chính xác với nhãn Schema	Bỏ qua Pha 3, trả về ngay không tốn gọi API

Bảng 2.4 Ví dụ minh họa kết quả Bước 3a — Top-3 ứng viên Schema

Relation thô và định nghĩa	Nhãn ứng viên (Cosine)	Kết quả Bước 3b
"làm co mạch khiến tăng huyết áp"	làm trầm trọng (0.921)	Ánh xạ
"không nên dùng cho người bệnh"	chống chỉ định với (0.888)	Ánh xạ
"được bác sĩ khuyến khích dùng"	được khuyến nghị cho (0.934)	Ánh xạ
"tăng nguy cơ mắc phải bệnh"	là yếu tố nguy cơ của (0.876)	Ánh xạ
"có mối liên hệ nhất định"	(F — None, max 0.51)	Loại bỏ

Bảng 2.5 Kết quả ánh xạ ngữ nghĩa và xác nhận quan hệ tại Pha 3

2.3.4 Xây Dựng và Chuẩn Hóa Đồ Thị Tri Thức

2.3.4.1 Hậu Xử Lý 2 Lớp

Sau khi hoàn thành 3 pha EDC, tập Triple thô vẫn còn một lượng nhiễu nhất định do LLM đôi khi trích xuất các Triple quá trừu tượng hoặc không chính xác về chiều quan hệ. Hệ thống áp dụng pipeline hậu xử lý 2 lớp:

Lớp 1 – Rule-based Cleaning (Lọc theo quy tắc):

Áp dụng 4 nhóm quy tắc lọc:

1. Loại bỏ Triple có Subject hoặc Object là khái niệm trừu tượng quá chung (VD: "chế độ ăn", "thực phẩm", "dinh dưỡng") không ánh xạ được về node cụ thể trong đồ thị.
2. Loại bỏ Triple có chiều quan hệ bị đảo ngược (VD: Disease → Food thay vì Food → Disease).
3. Loại bỏ Triple tự tham chiếu (Subject = Object).
4. Chuẩn hóa tên thực thể: viết thường, xóa dấu cách thừa, chuẩn hóa ký tự tiếng Việt.

Kết quả: Tỷ lệ nhiễu loại bỏ trung bình ~22% số Triple sau lớp 1.

Lớp 2 – Semantic Deduplication (Khử trùng lặp ngữ nghĩa):

Đây là bước tinh tế nhất. Do văn bản y khoa tiếng Việt có sự đa dạng từ ngữ rất cao (VD: "tiểu đường" / "đái tháo đường" / "đái tháo đường type 2" đều chỉ cùng một bệnh lý), lớp 2 dùng Jina Embeddings v3 để vector hóa tên của tất cả các thực thể, sau đó gom nhóm các thực thể có Cosine Similarity > 0.90 vào cùng một Node đại diện.

Công thức quyết định gom nhóm:

Nếu similarity(A, B) > 0.90: hợp nhất B vào A (giữ thực thể xuất hiện nhiều lần hơn làm tên chính thức).

Cập nhật tất cả Triple đang tham chiếu đến B, chuyển thành tham chiếu A.

Ví dụ gom nhóm thực tế:

"tiểu đường" + "đái tháo đường" + "đái tháo đường type 2" → Node duy nhất: "tiểu đường"

"tăng huyết áp" + "cao huyết áp" + "huyết áp cao" → Node duy nhất: "tăng huyết áp"

"suy thận" + "bệnh thận mãn tính" + "thận mãn" → Node duy nhất: "suy thận"

2.3.4.2 Import vào Neo4j và Chiến lược MERGE

Bước cuối cùng của pipeline là nạp toàn bộ Triple đã qua 2 lớp hậu xử lý vào Neo4j Graph Database. Câu lệnh Cypher sử dụng mệnh đề MERGE thay vì CREATE để đảm bảo tính idempotent — chạy import nhiều lần không tạo ra Node/Edge trùng lặp:

Kết quả cuối cùng của pipeline xây dựng Knowledge Graph:

- Tổng số Node: ~400 nodes (157 Food + 4 Disease + 21 Nutrient + ~237 Other)
- Tổng số Edge (Triple quan hệ): 2.000+ mối quan hệ y khoa đã chuẩn hóa
- Số loại quan hệ: 12 loại theo Schema chính thức
- Thời gian chạy pipeline (offline): Khoảng 45–60 phút cho toàn bộ bộ dữ liệu

2.4 TRIỂN KHAI VÀ DUY TRÌ

Hệ thống được triển khai theo mô hình Container-based Deployment thông qua Docker Compose, đảm bảo tính nhất quán giữa môi trường phát triển và vận hành.

Quy trình triển khai lần đầu:

1. **Chuẩn bị môi trường:** Cấu hình file .env với các biến môi trường (GROQ_API_KEY, JINA_KEY, NEO4J_PASSWORD).
2. **Khởi động container:** Chạy docker-compose up --build để build và khởi chạy đồng thời 4 dịch vụ (Neo4j, Backend, Frontend, Nginx).

3. **Nạp dữ liệu Knowledge Graph:** Chạy script rebuild_db.py để clear database Neo4j, sau đó lần lượt import 4 bộ Triple bệnh lý và 10 phần dữ liệu thực phẩm Việt Nam.
4. **Xác minh:** Truy cập Neo4j Browser (localhost:7474) để kiểm tra đồ thị, truy cập localhost để kiểm thử giao diện tư vấn.

Quy trình cập nhật dữ liệu (khi có bệnh lý hoặc thực phẩm mới):

Chuẩn bị tài liệu mới (.txt)

→ Chạy pipeline EDC (extract → postprocess)

→ Chạy import script tương ứng

→ Không cần rebuild container, không cần sửa code

Duy trì vận hành:

Quản lý API Keys: Theo dõi quota Groq (100,000 TPD Free Tier) và thay key định kỳ qua file .env.

Backup đồ thị: Sử dụng tính năng neo4j-admin dump để sao lưu định kỳ toàn bộ đồ thị.

Hot-reload Backend: FastAPI + Uvicorn với chế độ --reload, mọi thay đổi code tự động áp dụng mà không cần restart container.

2.5 PHƯƠNG PHÁP NGHIÊN CỨU

Đề tài áp dụng kết hợp đồng thời các phương pháp nghiên cứu sau:

a.Nghiên cứu lý thuyết và khảo sát tài liệu: Hệ thống hóa các lý thuyết nền tảng về Knowledge Graph, Open Information Extraction, Large Language Models và kiến trúc GraphRAG.Nghiên cứu sâu nguyên lý hoạt động của framework EDC (Extract-Define-Canonicalize) [1] và kỹ thuật Embedding bằng Jina v3 để hiểu cơ chế khử trùng lặp và biểu diễn ngữ nghĩa tiếng Việt.

b.Phương pháp thực nghiệm và đo lường: Xây dựng hệ thống theo phương pháp Agile/Iterative — phát triển theo từng vòng lặp ngắn, liên tục đo lường kết quả ở từng giai đoạn:

Đo tỷ lệ nhiễu trước/sau Rule-based Cleaning

Đo số Triple còn lại sau Semantic Deduplication ở các ngưỡng khác nhau (0.85, 0.90, 0.95)

Đo thời gian phản hồi API theo điều kiện có/không có cache

c.Phương pháp Prompt Engineering: Thiết kế và tối ưu hóa có hệ thống các mẫu lệnh (few-shot prompt templates) cho từng module trong pipeline EDC.Áp dụng kỹ thuật Chain-of-Thought và Role-prompting để cải thiện độ chính xác của quá trình trích xuất Triple từ văn bản y khoa tiếng Việt.

d.Phương pháp đánh giá định tính: Kiểm thử hộp đen (black-box testing) bằng cách nhập đa dạng các trường hợp đặc biệt: món ăn phổ biến, món ăn lạ, tên viết tắt, từ tiếng Anh xen tiếng Việt...để đánh giá tổng thể độ tin cậy của hệ thống tư vấn y khoa.

CHƯƠNG 3: GIỚI THIỆU TỔNG QUAN VỀ VẤN ĐỀ NGHIÊN CỨU

3.1 TỔNG QUAN VỀ VẤN ĐỀ NGHIÊN CỨU

3.1.1 Khái niệm cơ bản

Để hiểu rõ hệ thống được xây dựng trong đề tài này, cần nắm vững một số khái niệm nền tảng sau:

a.Đồ thị Tri thức (Knowledge Graph – KG)

Đồ thị Tri thức là một cấu trúc dữ liệu dạng mạng ngữ nghĩa, trong đó tri thức được biểu diễn dưới dạng các bộ ba (Triple): (Thực thể A, Quan hệ, Thực thể B).Ví dụ:

(thịt đỏ, là yếu tố nguy cơ của, suy thận)

(muối, cần hạn chế ở, tăng huyết áp)

(rau xanh, tốt cho, béo phì)

KG không chỉ lưu trữ dữ liệu như bảng thông thường mà còn mã hóa mối liên hệ ngữ nghĩa giữa các thực thể, cho phép các hệ thống AI suy luận và tìm kiếm theo ngữ cảnh.

b.Trích xuất Thông tin Mở (Open Information Extraction – OIE)

OIE là kỹ thuật tự động trích xuất các Triple có cấu trúc từ văn bản tự do, không phụ thuộc vào một tập quan hệ định nghĩa trước.Cụ thể trong đề tài này, mô hình Llama 3.3 được điều khiển bằng cơ chế few-shot prompting để đọc đoạn văn y khoa tiếng Việt và xuất ra danh sách các bộ ba tri thức.

c.Framework EDC (Extract – Define – Canonicalize) [1]

EDC là khung phương pháp luận hệ thống cho quá trình trích xuất và chuẩn hóa tri thức từ văn bản tự do:

Extract: Trích xuất Triple thô bằng OIE và LLM

Define: Từ tập Triple thô, hệ thống tự đề xuất tập hợp các nhãn quan hệ (Relation Schema) đặc thù của miền kiến thức

Canonicalize: Ánh xạ các quan hệ có cách diễn đạt khác nhau về cùng một nhãn chuẩn bằng Embedding Similarity

d.GraphRAG (Graph Retrieval-Augmented Generation)

GraphRAG là kiến trúc kết hợp hai thành phần:

Graph Retrieval: Truy vấn cơ sở dữ liệu đồ thị (Neo4j) để lấy các ngữ cảnh tri thức liên quan đến câu hỏi của người dùng

Augmented Generation: Đưa ngữ cảnh đó vào prompt của LLM để sinh ra câu trả lời có căn cứ, tránh ảo giác (hallucination)

e.Semantic Embedding (Nhúng ngữ nghĩa)

Là kỹ thuật biến đổi văn bản thành vector số học trong không gian nhiều chiều, sao cho các văn bản có nghĩa tương đồng sẽ có vector gần nhau (Cosine Similarity cao).Trong đề tài, Jina Embeddings v3 được dùng cho cả hai tác vụ: (1) Chuẩn hóa Schema trong pipeline EDC, và (2) Khử trùng lặp thực thể trong bước hậu xử lý Knowledge Graph.

f. Phân Đoạn Văn Bản (Text Chunking)

Text Chunking là kỹ thuật chia nhỏ tài liệu dài thành các đoạn (chunk) có kích thước phù hợp trước khi đưa vào xử lý bởi LLM. Kỹ thuật này xuất phát từ giới hạn Context Window của các mô hình ngôn ngữ lớn — tức là giới hạn về số lượng token có thể được xử lý trong một lần gọi API.

Trong đề tài, chiến lược chunking được lựa chọn dựa trên đặc thù của văn bản y khoa: cắt theo ranh giới đoạn văn (paragraph boundary) với kích thước mỗi chunk từ 800 đến 1.200 ký tự. Cách tiếp cận này đảm bảo mỗi chunk chứa đựng một ý nghĩa chủ đề nhất quán, giúp LLM trích xuất Triple chính xác hơn so với việc cắt theo số ký tự cứng nhắc.

g. Kỹ thuật Few-Shot Prompting trong Trích xuất Thông tin

Few-Shot Prompting (Học từ vài ví dụ) là kỹ thuật cung cấp cho LLM một số lượng nhỏ ví dụ mẫu (thường từ 3–10 cặp Input/Output) ngay trong câu lệnh (prompt) để định hướng hành vi của mô hình, mà không cần phải tinh chỉnh (fine-tune) lại tham số mô hình. Kỹ thuật này đặc biệt hiệu quả trong bài toán Trích xuất Thông tin Có Cấu Trúc (Structured IE) vì:

- Cho phép định nghĩa chính xác định dạng đầu ra mong muốn (dạng Triple).
- Giúp LLM học được ngữ cảnh chuyên ngành (y khoa dinh dưỡng) chỉ từ vài ví dụ, không cần bộ dữ liệu gán nhãn hàng nghìn mẫu.
- Dễ điều chỉnh (chỉ cần thay đổi file few-shot examples) khi cần mở rộng sang miền kiến thức mới.

Trong đề tài, 10 few-shot examples được thiết kế thủ công để bao phủ đủ các loại quan hệ dinh dưỡng–bệnh lý theo hai hướng (tích cực và tiêu cực), giúp mô hình Llama 3.3-70B tạo ra Triple chất lượng cao ngay từ lần gọi đầu tiên.

h. Độ Tương Đồng Cosine và Ngưỡng Quyết Định trong Semantic Matching

Độ tương đồng Cosine (Cosine Similarity) là chỉ số đo mức độ tương đồng ngữ nghĩa giữa hai vector embedding, được tính theo công thức:

$$\text{cosine_similarity}(A, B) = \frac{A \cdot B}{|A| \times |B|}$$

Kết quả nằm trong khoảng [-1, 1], trong đó giá trị 1 có nghĩa là hai vector hoàn toàn trùng hướng (cùng nghĩa), giá trị 0 là không liên quan, và giá trị -1 là đối nghịch hoàn toàn.

Trong đề tài, ngưỡng Cosine Similarity được áp dụng cho hai tác vụ khác nhau:

- **Ngưỡng 0.75 cho Schema Canonicalization:** Một Relation thô được ánh xạ về nhãn Schema nếu có Cosine Similarity ≥ 0.75 với định nghĩa nhãn đó. Ngưỡng này đủ rộng để bắt được các cách diễn đạt đồng nghĩa nhưng không quá thấp để gây nhầm lẫn giữa các nhãn quan hệ có nghĩa gần nhau.
- **Ngưỡng 0.90 cho Semantic Deduplication:** Hai thực thể được gom vào cùng một Node nếu Cosine Similarity ≥ 0.90. Ngưỡng cao hơn được áp dụng ở đây vì việc gom nhầm hai bệnh lý khác nhau (ví dụ: "tiểu đường" và "tăng huyết áp") sẽ gây lỗi nghiêm trọng trong đồ thị.

3.1.2 Mục tiêu của hệ thống Xây dựng đồ thị tri thức dinh dưỡng bệnh nhân

Hệ thống "Xây dựng Đồ thị Tri thức Dinh dưỡng Bệnh nhân" được xây dựng nhằm đạt đồng thời các mục tiêu sau:

Mục tiêu 1 – Tự động hóa số hóa tri thức y khoa: Thay vì phải thuê chuyên gia y tế ngồi nhập liệu thủ công, hệ thống tự động trích xuất Triple từ văn bản y khoa tiếng Việt bằng pipeline EDC.Điều này cho phép xây dựng Knowledge Graph quy mô lớn với chi phí nhân công tối thiểu.

Mục tiêu 2 – Xây dựng nền tảng dữ liệu dinh dưỡng toàn diện: Kết hợp hai nguồn dữ liệu bổ sung lẫn nhau trong Neo4j: (1) Quan hệ ngữ nghĩa y khoa từ Triple KG (thực phẩm nào có hại/có lợi cho bệnh gì), và (2) Số liệu định lượng dinh dưỡng từ bảng thành phần thực phẩm Việt Nam (16 vi chất đo lường được).

Mục tiêu 3 – Cung cấp tư vấn dinh dưỡng chính xác và cá nhân hóa: Dựa trên bệnh lý người dùng khai báo, hệ thống truy vấn đồ thị và tổng hợp lời khuyên riêng biệt cho từng cặp (Thực phẩm, Bệnh lý), đảm bảo thông tin luôn bám sát dữ liệu y khoa đã xác minh.

Mục tiêu 4 – Đảm bảo an toàn thông tin y tế: Triển khai cơ chế Circuit Breaker — hệ thống sẽ từ chối sinh lời khuyên và hiển thị cảnh báo rõ ràng khi thực phẩm không

có trong cơ sở tri thức, thay vì để mô hình ngôn ngữ lớn (LLM) sinh ra các thông tin sai lệch (ảo giác AI).

3.1.3 Phạm vi ứng dụng

Hệ thống được thiết kế phục vụ cho các đối tượng sau:

Người bệnh mãn tính và gia đình: Đối tượng chính cần hỗ trợ nhất.Người mắc một trong bốn nhóm bệnh (Tiểu đường, Tăng huyết áp, Bệnh thận mãn tính, Béo phì) có thể sử dụng ứng dụng trực tiếp để kiểm tra ngay lập tức một món ăn có phù hợp với tình trạng sức khỏe của mình hay không trước khi ăn.

Nhân viên y tế và điều dưỡng: Có thể dùng như công cụ tra cứu nhanh bên cạnh hướng dẫn lâm sàng, hỗ trợ tư vấn dinh dưỡng sơ bộ cho bệnh nhân tại phòng khám hoặc tại nhà.

Nhà nghiên cứu và sinh viên ngành dinh dưỡng: Hệ thống cung cấp nền tảng để khám phá các mối quan hệ trong đồ thị tri thức dinh dưỡng, phục vụ mục đích học tập và nghiên cứu.

Giới hạn phạm vi: Hệ thống không thay thế chẩn đoán y khoa chính thức.Mọi lời khuyên từ hệ thống chỉ mang tính tham khảo và phải được xác nhận bởi bác sĩ chuyên khoa trước khi áp dụng.

3.1.4 Khả năng phát triển

Kiến trúc mô-đun của hệ thống tạo nền tảng vững chắc cho nhiều hướng mở rộng trong tương lai:

Mở rộng tri thức theo chiều ngang (Thêm bệnh lý): Do toàn bộ tri thức lưu trữ trong Neo4j tách biệt hoàn toàn với code, việc bổ sung một nhóm bệnh mới (ví dụ: Gout, Ung thư, Tim mạch) chỉ cần chuẩn bị tài liệu y khoa và chạy lại pipeline EDC rồi import — không cần sửa bất kỳ dòng code nào ở Backend hay Frontend.

Cá nhân hóa theo hồ sơ người dùng: Xây dựng sub-graph riêng lưu lịch sử bữa ăn, xét nghiệm sinh hóa và mục tiêu sức khỏe của mỗi người dùng.LLM sẽ tư vấn chủ động dựa trên bức tranh đầy đủ thay vì từng câu hỏi đơn lẻ.

Tích hợp Knowledge Graph Embedding (KGE): Ứng dụng các phương pháp học biểu diễn đồ thị (TransE, RotatE) để suy luận ra các quan hệ ẩn (link prediction), ví dụ: tự động dự đoán rằng một thực phẩm mới có thể có hại cho một nhóm bệnh nhất định dựa trên thành phần dinh dưỡng [9] của nó.

Triển khai lên đám mây: Kiến trúc Docker Compose hiện tại có thể chuyển đổi dễ dàng sang Docker Swarm hoặc Kubernetes, cho phép scale-out khi hệ thống có nhiều người dùng đồng thời.

3.1.5 Các yêu cầu hệ thống

Yêu cầu phần cứng tối thiểu (để vận hành):

Thành phần	Yêu cầu tối thiểu	Ghi chú
CPU	4 cores	Neo4j và FastAPI đều CPU-bound
RAM	8 GB	Neo4j cần ≥ 4GB heap, FastAPI cần ~1GB
Ổ đĩa	10 GB SSD	Cho Docker images + Neo4j data
GPU	Không bắt buộc	Toàn bộ Embedding và LLM dùng Cloud API
Kết nối mạng	Cần có Internet	Truy cập Groq API và Jina Embeddings API

Bảng 3.1 Bảng yêu cầu phần cứng tối thiểu

Yêu cầu phần mềm:

Phần mềm	Phiên bản tối thiểu
Docker Desktop	4.20+
Docker Compose	v2.0+
Python (cho pipeline ngoài)	3.10+

Bảng 3.2 Bảng yêu cầu phần mềm

Yêu cầu API Keys (bắt buộc):

Biến môi trường	Dịch vụ	Mục đích
GROQ_API_KEY	Groq Cloud	Gọi Llama 3.3 (tư vấn) + Llama 4 Scout (nhận diện ảnh)
JINA_KEY	Jina AI	Jina Embeddings v3 (Schema Canonicalization + Khử trùng lặp)

Bảng 3.3 Bảng Yêu cầu API Keys

3.2 KHẢO SÁT HIỆN TRẠNG

3.2.1 Các hệ thống tương tự hiện có

a.Ứng dụng tư vấn dinh dưỡng thông thường (Nutrition Apps):

Thị trường hiện có nhiều ứng dụng như MyFitnessPal, Cronometer hay các ứng dụng dinh dưỡng của Viện Dinh dưỡng Quốc gia Việt Nam.Các ứng dụng này cung cấp

bảng tra cứu thành phần thực phẩm và tính toán calo theo mục tiêu cá nhân.Tuy nhiên, chúng hoàn toàn không có khả năng tư vấn y khoa theo bệnh lý — chúng không phân biệt người dùng có tiểu đường hay suy thận với người bình thường.

b.Chatbot y tế tổng quát (General Medical Chatbots):

Các trợ lý ảo như ChatGPT hay Google Gemini có khả năng trả lời câu hỏi y tế bằng tiếng Việt.Tuy nhiên, chúng dựa hoàn toàn vào kiến thức đã học trong quá trình huấn luyện mà không có cơ chế kiểm chứng — dẫn đến nguy cơ ảo giác (hallucination) rất cao trong các tình huống đòi hỏi độ chính xác y khoa tuyệt đối.

c.Hệ thống Knowledge Graph y tế lớn (Medical KGs):

Ở quy mô quốc tế, có các KG y tế như UMLS (Unified Medical Language System), DrugBank hay các đồ thị dinh dưỡng của USDA.Tuy nhiên, các hệ thống này hoàn toàn không phù hợp với thực phẩm Việt Nam (thịt kho tàu, rau muống, cơm tấm...) và không có giao diện tư vấn tiếng Việt cho người dùng cuối.

3.2.2 Phân tích khoảng trống và lý do cần giải pháp mới

So sánh tổng hợp các giải pháp hiện có:

Tiêu chí	**Nutrition Apps**	**ChatGPT / Gemini**	**Medical KGs nước ngoài**	**Hệ thống đề tài**
Thực phẩm Việt Nam	Có	Một phần	Không	Có (150+ món)
Tư vấn theo bệnh lý	Không	Có nhưng không kiểm chứng	Có	Có (bốn nhóm bệnh)
Chống ảo giác AI	N/A	Không	N/A	Circuit Breaker
Giải thích nguồn gốc	Không	Không	Có	Đồ thị có thể truy vết
Nhận diện món ăn từ ảnh	Không	Giới hạn	Không	Vision AI (Llama 4)
Giao diện tiếng Việt	Một số	Có	Không	Có (tiếng Việt đầy đủ)
Mã nguồn mở / Tùy biến	Không	Không	Một phần	Hoàn toàn

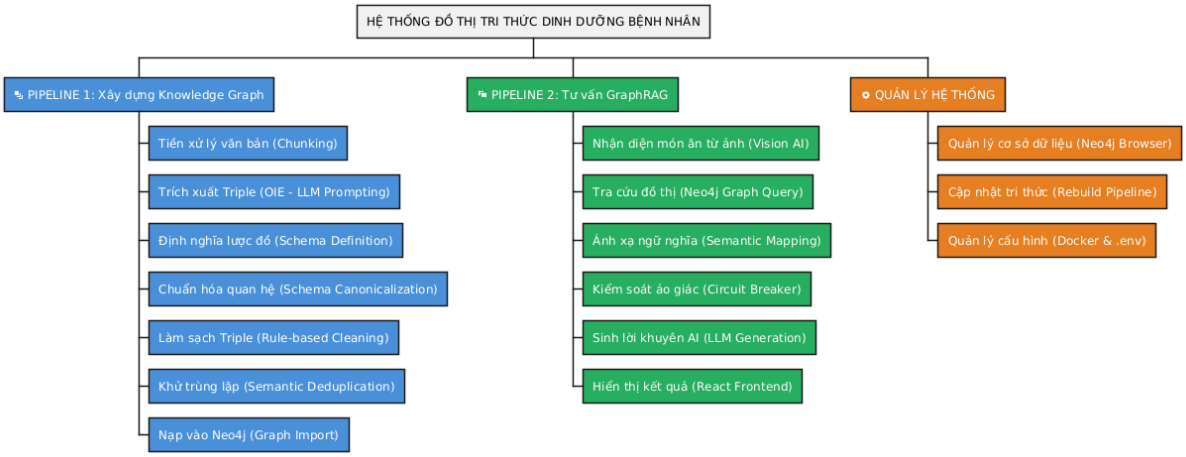
Bảng 3.4 Bảng so sánh tổng hợp các giải pháp hiện có

Kết luận khảo sát:

Qua khảo sát, có thể thấy rõ rằng chưa có hệ thống nào trên thị trường Việt Nam giải quyết đồng thời được bốn yêu cầu cốt lõi: (1) cơ sở dữ liệu thực phẩm Việt Nam phong phú, (2) tư vấn theo bệnh lý mãn tính đặc thù, (3) cơ chế đảm bảo an toàn thông tin y tế (chống ảo giác), và (4) tri thức có cấu trúc, có thể truy vết nguồn gốc.Đây chính là khoảng trống mà đề tài "Xây dựng Đồ thị Tri thức Dinh dưỡng Bệnh nhân" hướng đến lấp đầy.

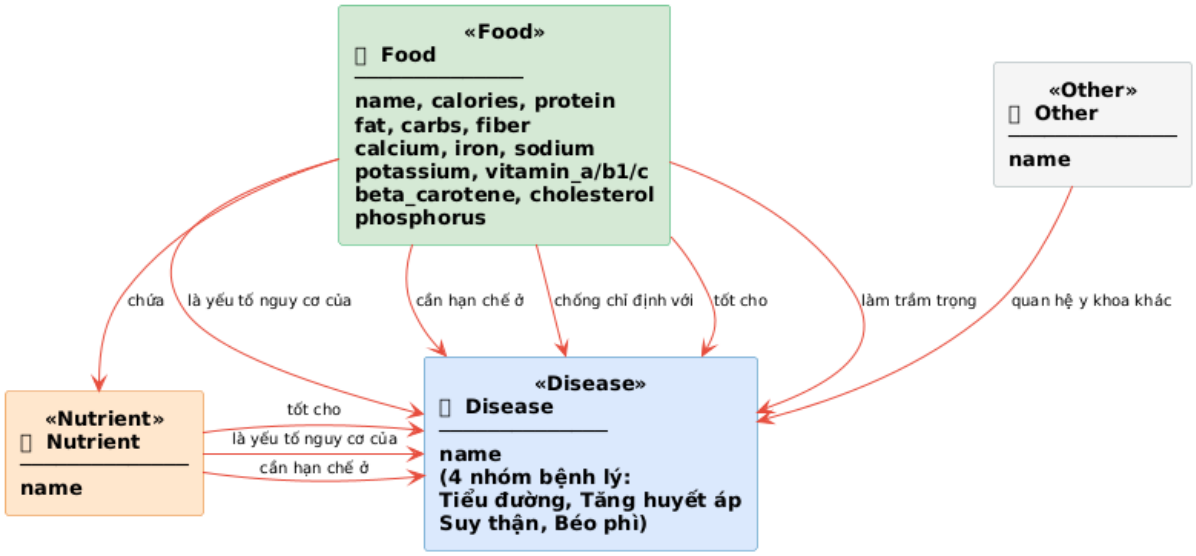
CHƯƠNG 4: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG BẰNG NGÔN NGỮ UML

4.1 MÔ HÌNH PHÂN RÃ CHỨC NĂNG (Business Function Diagram – BFD)



Hình 4.1 Mô hình phân rã chức năng

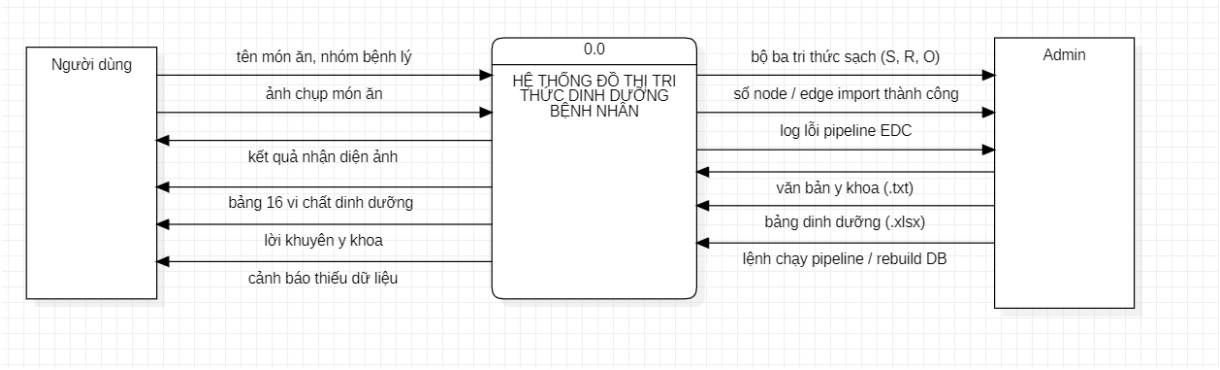
4.2 MÔ HÌNH QUAN HỆ THỰC THỂ - SCHEMA ĐỒ THỊ TRI THỨC (KG Schema)



Hình 4.2 Mô hình quan hệ thực thể

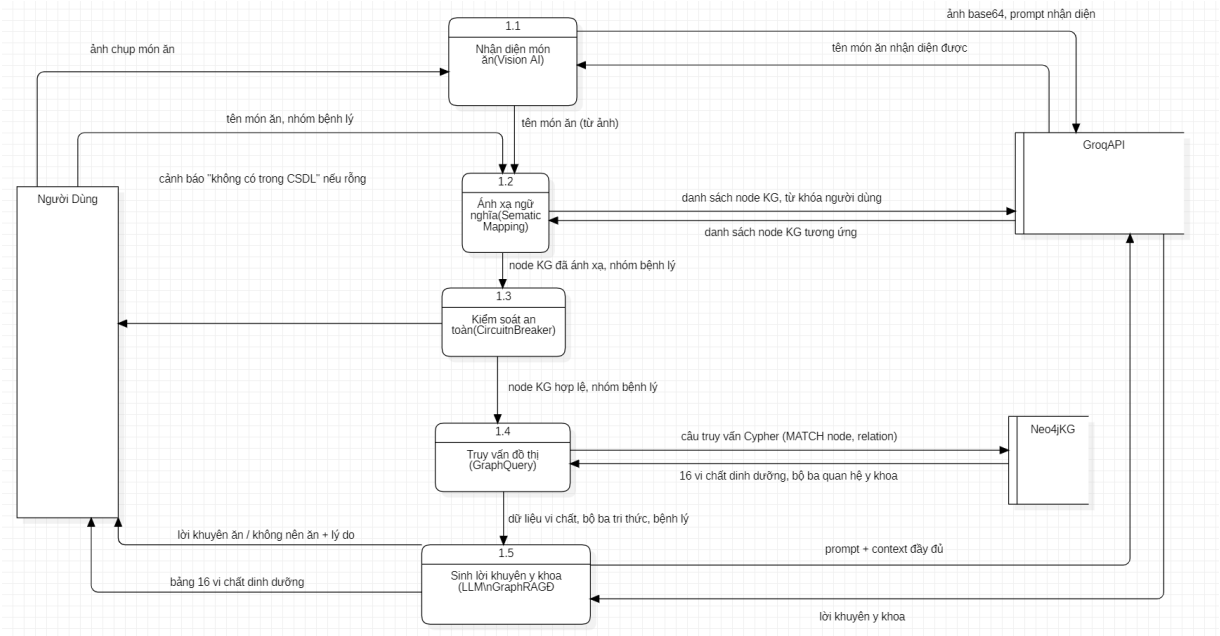
4.3 MÔ HÌNH LUỒNG DỮ LIỆU (Data Flow Diagram – DFD)

4.3.1 Mức F0 Người Dùng/Admin



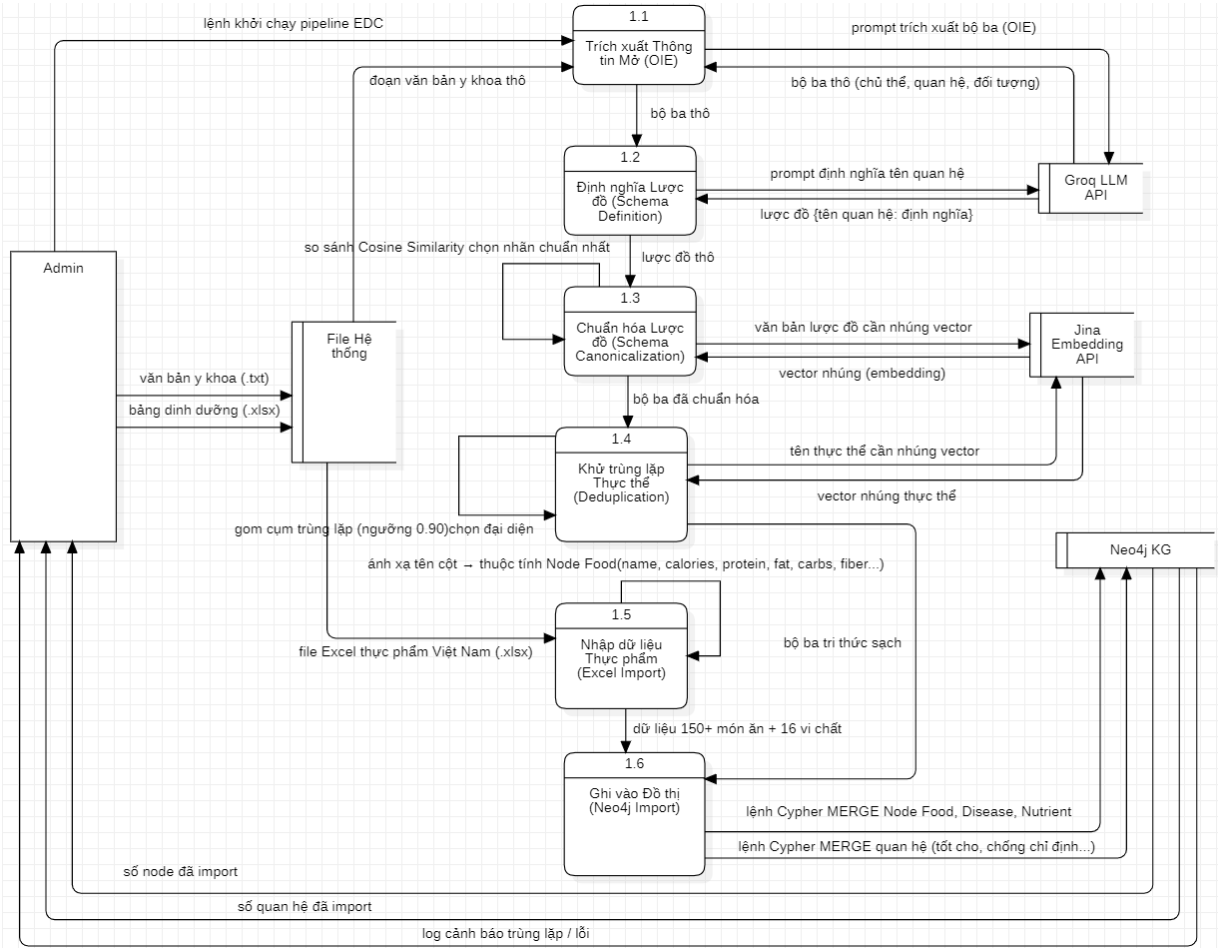
Hình 4.3 Mức F0 Người Dùng và Admin

4.3.2 Mức F1 Người Dùng



Hình 4.4 Mức F1 Người Dùng

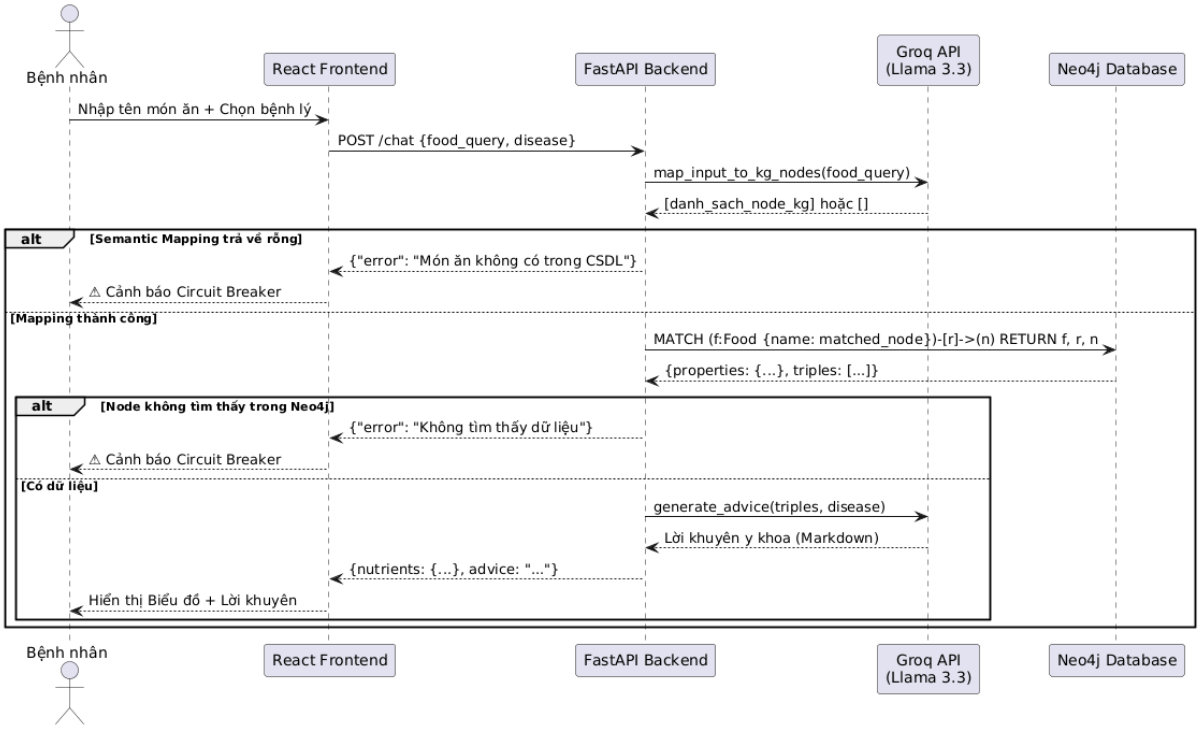
4.3.3 Mức F1 Admin



Hình 4.5 Mức F1 Admin

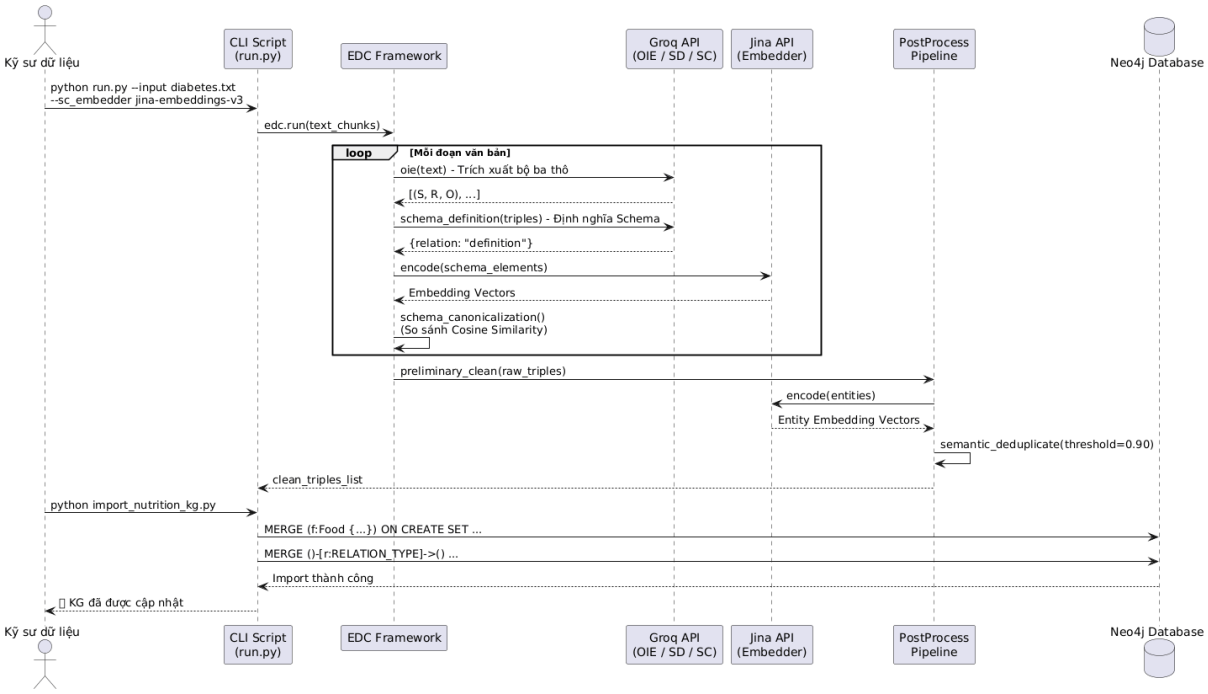
4.4 MÔ HÌNH SƠ ĐỒ TUẦN TỰ (Sequence Diagram)

4.4.1 Luồng Tra cứu Dinh dưỡng (GraphRAG)



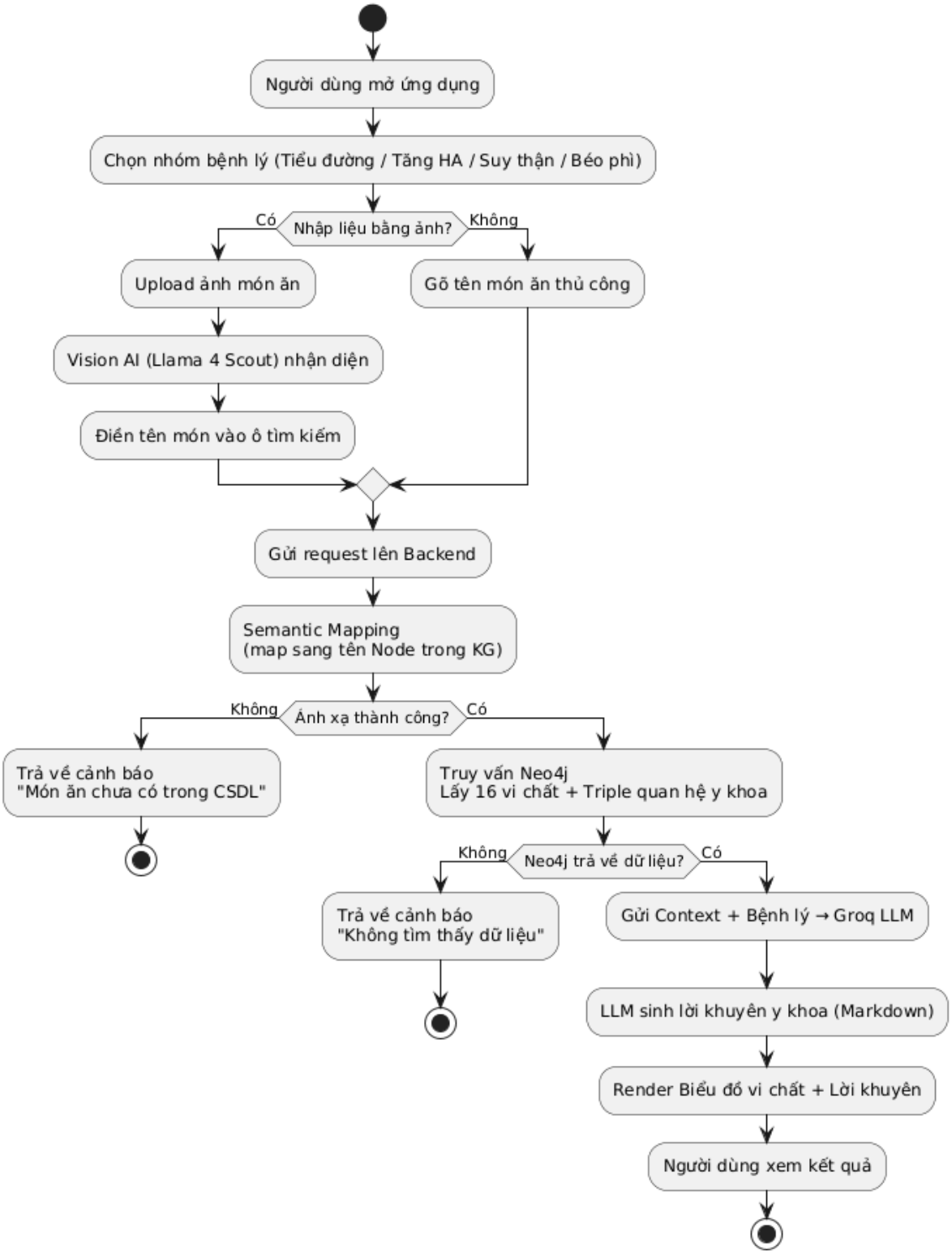
Hình 4.6 Luồng Tra cứu Dinh dưỡng

4.4.2 Pipeline Xây dựng KG (Admin/EDC)



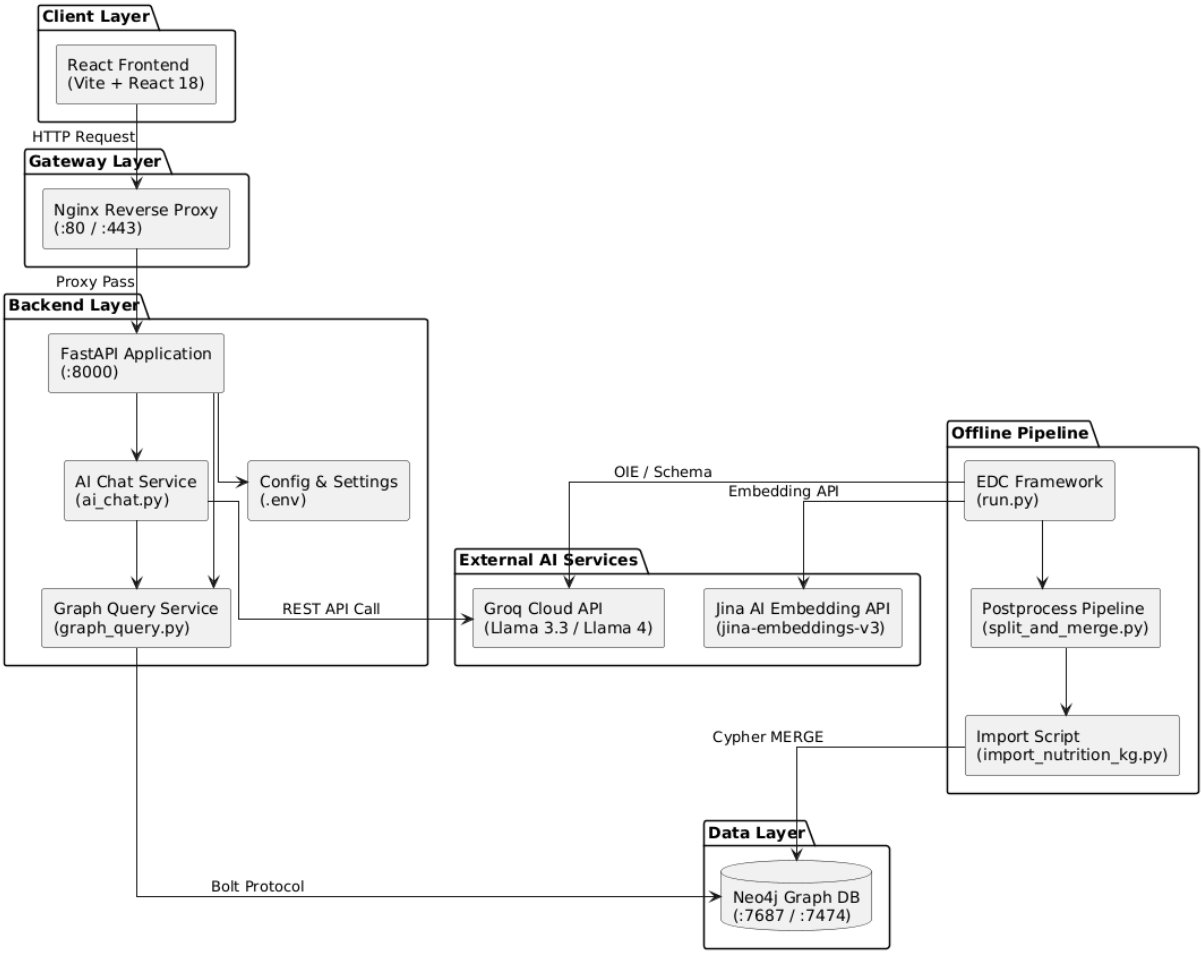
Hình 4.7 Pipeline Xây dựng KG

4.5 MÔ HÌNH SƠ ĐỒ HOẠT ĐỘNG (Activity Diagram)



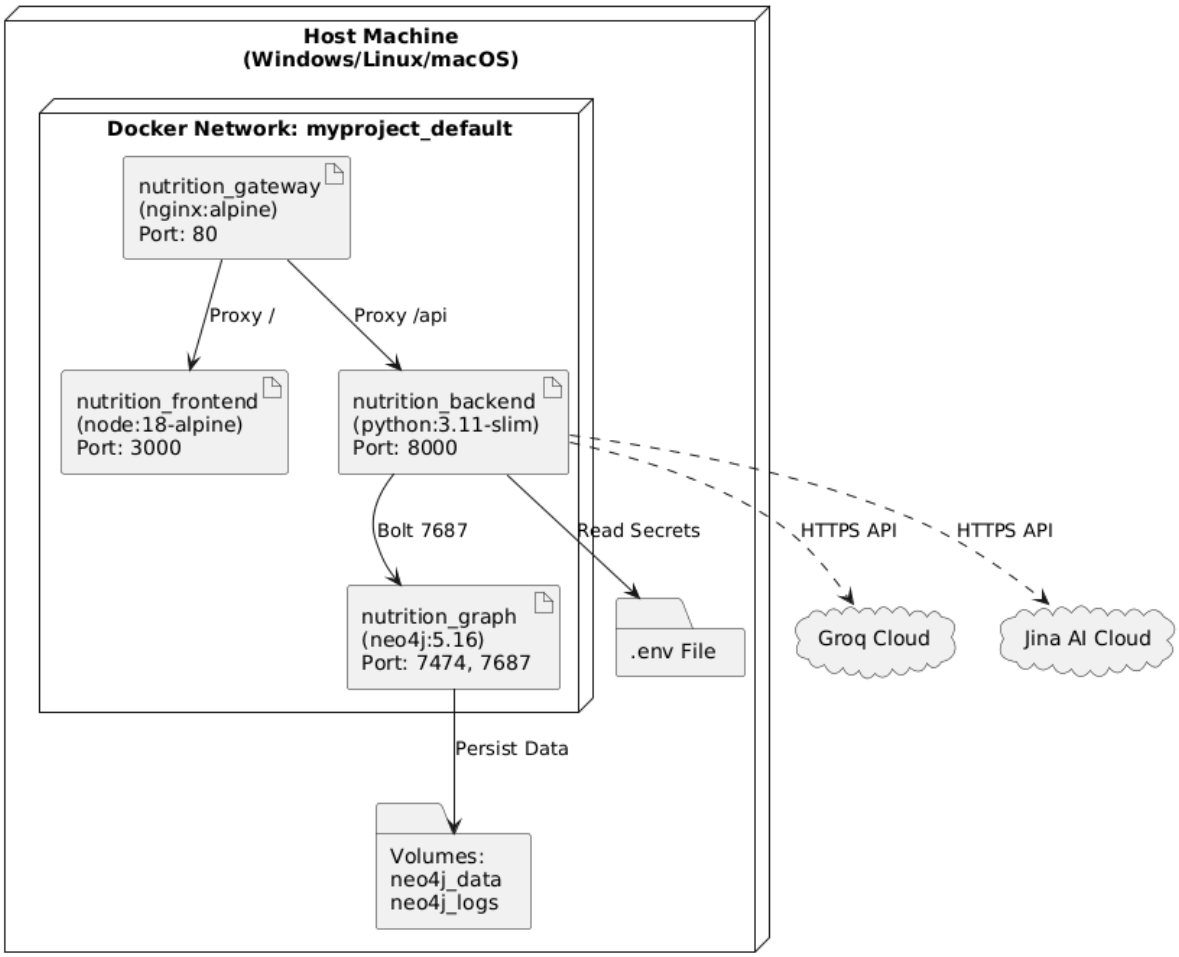
Hình 4.8 Mô hình sơ đồ hoạt động

4.6 MÔ HÌNH KIẾN TRÚC THÀNH PHẦN HỆ THỐNG (Component Diagram).



Hình 4.9 Mô hình kiến trúc thành phần hệ thống

4.7 MÔ HÌNH SƠ ĐỒ TRIỂN KHAI (Deployment Diagram)



Hình 4.10 Sơ đồ triển khai

CHƯƠNG 5: THIẾT KẾ CƠ SỞ DỮ LIỆU VÀ USE CASE

5.1 BẢNG CƠ SỞ DỮ LIỆU (Từ điển dữ liệu Entity)

STT	Tên thuộc tính	Kiểu dữ liệu	Mô tả
1	name	String	Tên món ăn (Unique Constraint)
2	food_group	String	Nhóm phân loại (VD: Rau củ, Thịt...)
3	calories	Float	Năng lượng (kcal trên 100g)
4	protein	Float	Lượng đạm (g)
5	fat	Float	Lượng chất béo (g)
6	carbs	Float	Lượng đường bột (g)
7	fiber	Float	Lượng chất xơ (g)
8	cholesterol	Float	Lượng cholesterol (mg)
9	calcium	Float	Lượng canxi (mg)
10	phosphorus	Float	Lượng phospho (mg)
11	iron	Float	Lượng sắt (mg)
12	sodium	Float	Lượng natri (mg)
13	potassium	Float	Lượng kali (mg)
14	beta_carotene	Float	Lượng Beta-carotene (mcg)
15	vitamin_a	Float	Lượng Vitamin A (mcg)
16	vitamin_b1	Float	Lượng Vitamin B1 (mg)
17	vitamin_c	Float	Lượng Vitamin C (mg)

Bảng 5.1 Bảng chi tiết thuộc tính thực thể Food (Thực phẩm)

Nhãn (Label)	Thuộc tính	Kiểu dữ liệu	Mô tả	Khóa
Disease	name	String	Tên bệnh lý (VD: "tiểu đường")	Unique
Nutrient	name	String	Tên dưỡng chất (từ NLP KG)	Unique
Other	name	String	Khái niệm y khoa chung khác	Unique

Bảng 5.2 Bảng cấu trúc Nodes

5.2 BẢNG PHÁT HIỆN CÁC RÀNG BUỘC CƠ SỞ DỮ LIỆU (Constraints & Relationships)

5.2.1 Ràng buộc toàn vẹn (Graph Constraints)

- CONSTRAINT UniqueFoodName: Cột name của nhãn Food là duy nhất.
- CONSTRAINT UniqueDiseaseName: Cột name của nhãn Disease là duy nhất.
- CONSTRAINT UniqueNutrientName: Cột name của nhãn Nutrient là duy nhất.
- CONSTRAINT UniqueOtherName: Cột name của nhãn Other là duy nhất.

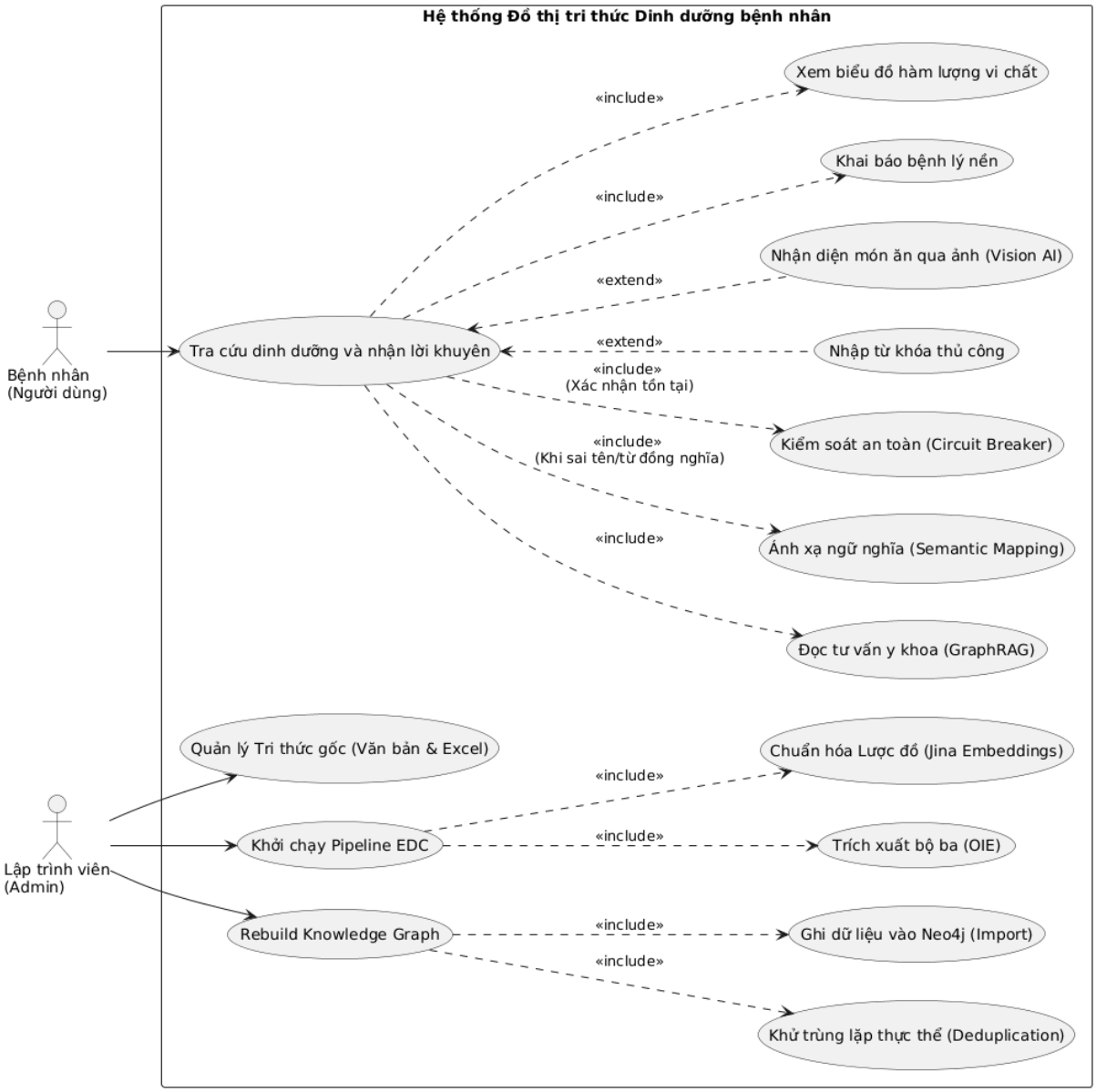
5.2.2 Cấu trúc dữ liệu Quan hệ (Edge Schema)

Tên Quan hệ (Relation Type)	Node Nguồn (Start)	Node Đích (End)	Ý nghĩa y khoa
làm trầm trọng	Food / Nutrient	Disease	Thực phẩm/dưỡng chất làm xấu đi triệu chứng hoặc thúc đẩy tiến triển bệnh
được khuyến nghị cho	Food / Nutrient	Disease	Được chuyên gia y tế khuyến nghị cho bệnh nhân mắc bệnh này
cần hạn chế ở	Food / Nutrient	Disease	Cần giảm lượng tiêu thụ ở bệnh nhân mắc bệnh
chống chỉ định với	Food	Disease	Gây hại và cần tránh hoàn toàn ở bệnh nhân
phòng ngừa	Food / Nutrient	Disease	Giúp ngăn ngừa bệnh hoặc biến chứng khi tiêu thụ đầy đủ

Tên Quan hệ (Relation Type)	**Node Nguồn (Start)**	**Node Đích (End)**	**Ý nghĩa y khoa**
hỗ trợ	Food / Nutrient	Disease	Hỗ trợ kiểm soát hoặc cải thiện tình trạng bệnh
ảnh hưởng đường huyết	Food	Disease / Other	Ảnh hưởng mức đường huyết theo chiều hướng: tăng / tăng nhẹ / giảm
giàu	Food	Nutrient	Thực phẩm là nguồn giàu dưỡng chất, hàm lượng cao hơn mức trung bình
ít	Food	Nutrient	Thực phẩm có hàm lượng thấp của chỉ số dinh dưỡng
thiếu hụt gây ra	Nutrient	Disease	Thiếu hụt dưỡng chất dẫn đến bệnh hoặc tình trạng sức khỏe
tương tác với	Nutrient	Nutrient / Other	Tương tác — tăng cường hoặc cản trở hấp thu/chuyển hóa
là yếu tố nguy cơ của	Food / Nutrient	Disease	Làm tăng đáng kể nguy cơ mắc bệnh

Bảng 5.3 Bảng cấu trúc dữ liệu quan hệ

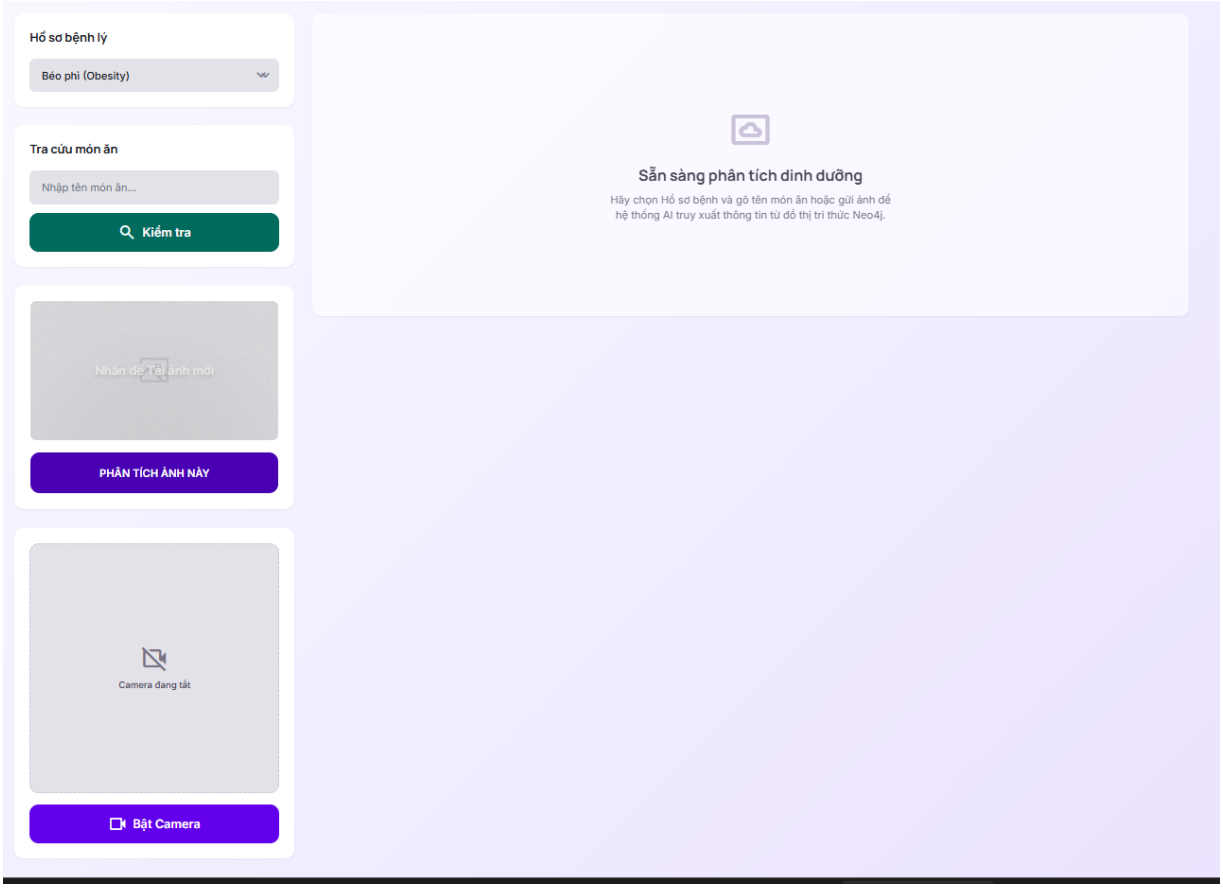
5.3 SƠ ĐỒ USE CASE



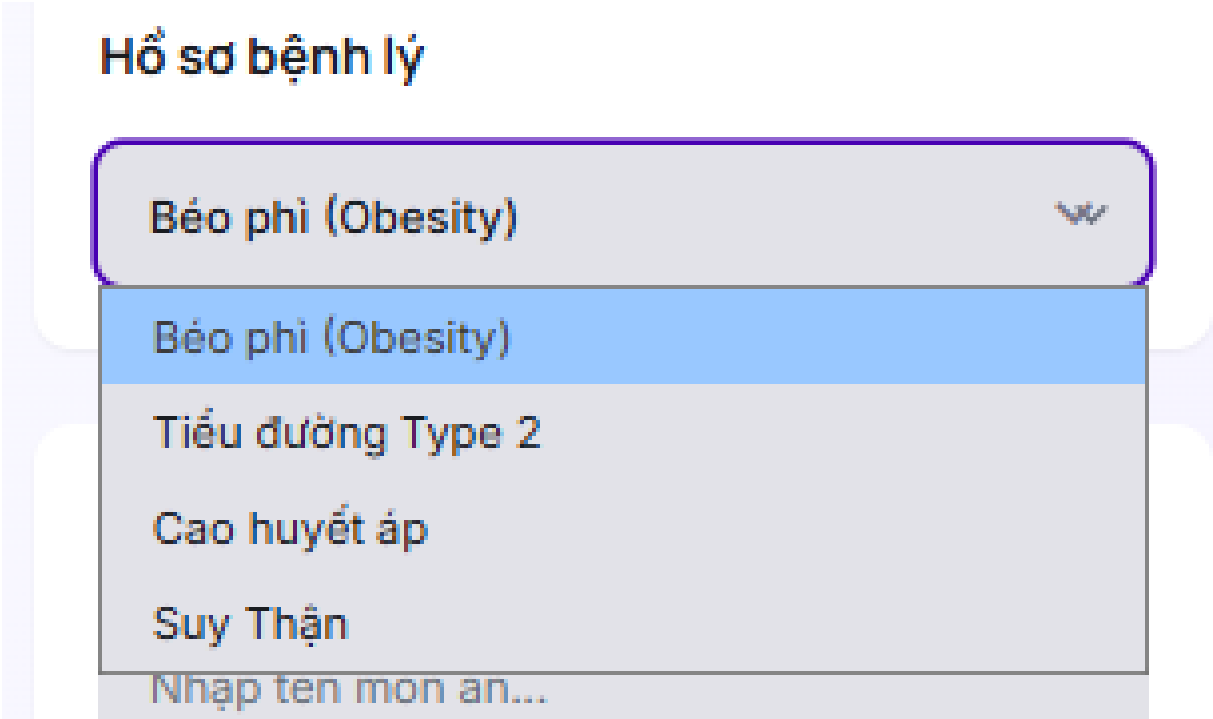
Hình 5.1 Sơ đồ Use Case

CHƯƠNG 6: ĐẶC TẢ GIAO DIỆN

6.1 GIAO DIỆN TRANG CHỦ VÀ KHAI BÁO BỆNH LÝ (Home & Input Panel)

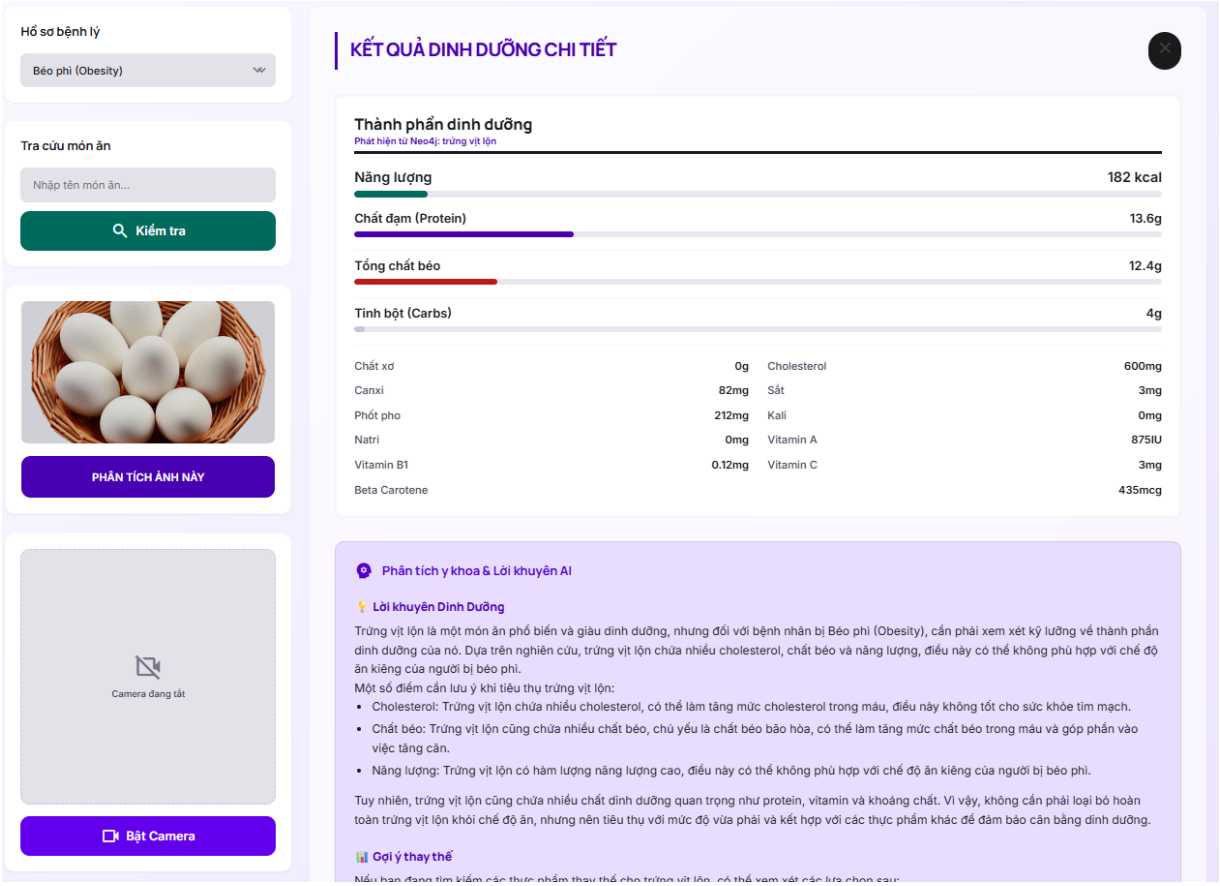


Hình 6.1 Giao diện trang chủ



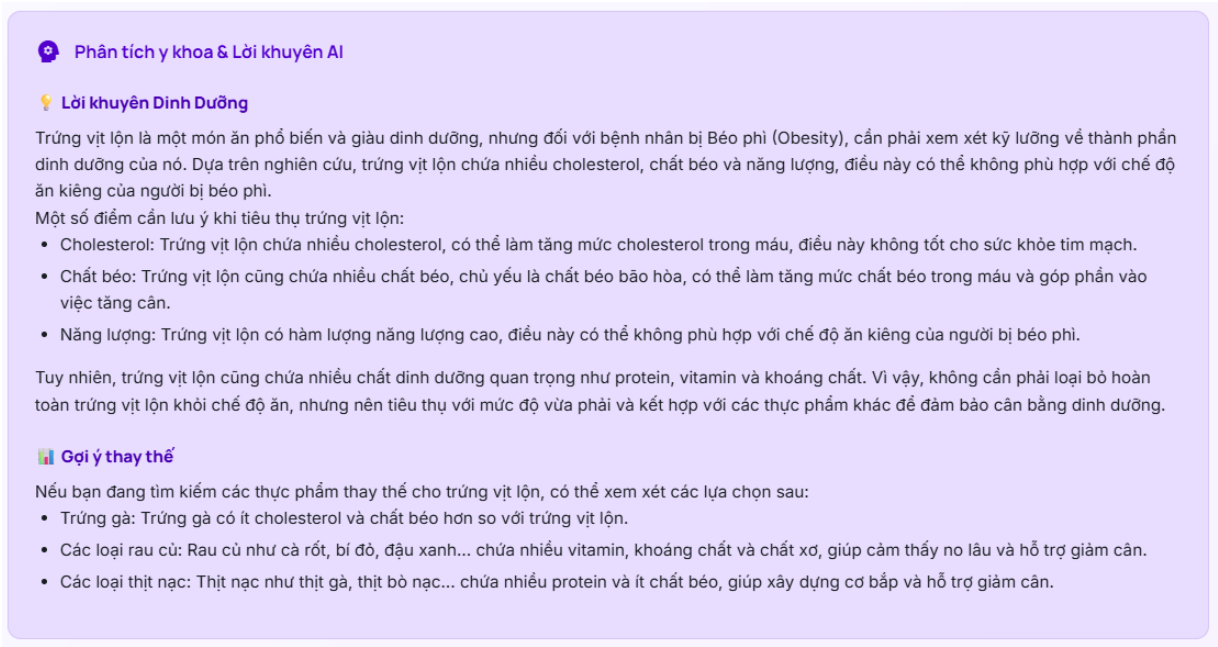
Hình 6.2 Khai báo bệnh lý

6.2 GIAO DIỆN NHẬN DIỆN MÓN ĂN AI (Vision Upload Panel)



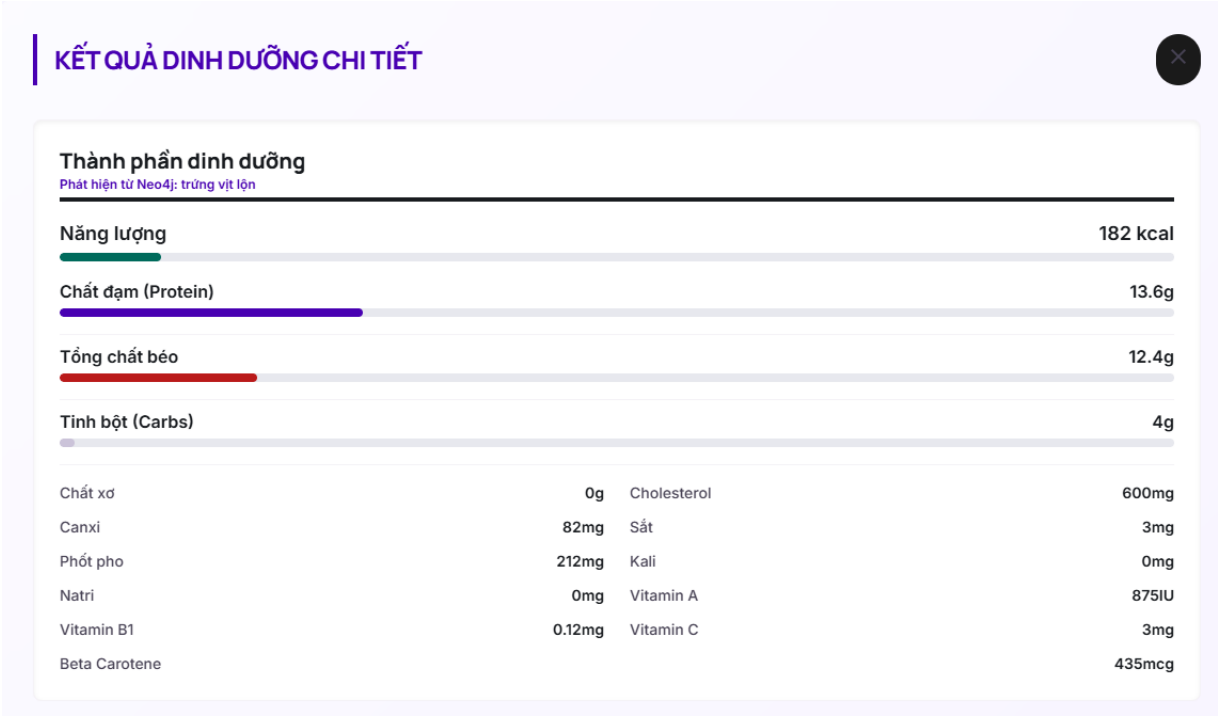
Hình 6.3 Nhận diện món ăn

6.3 GIAO DIỆN TƯ VẤN Y KHOA (GraphRAG Output)



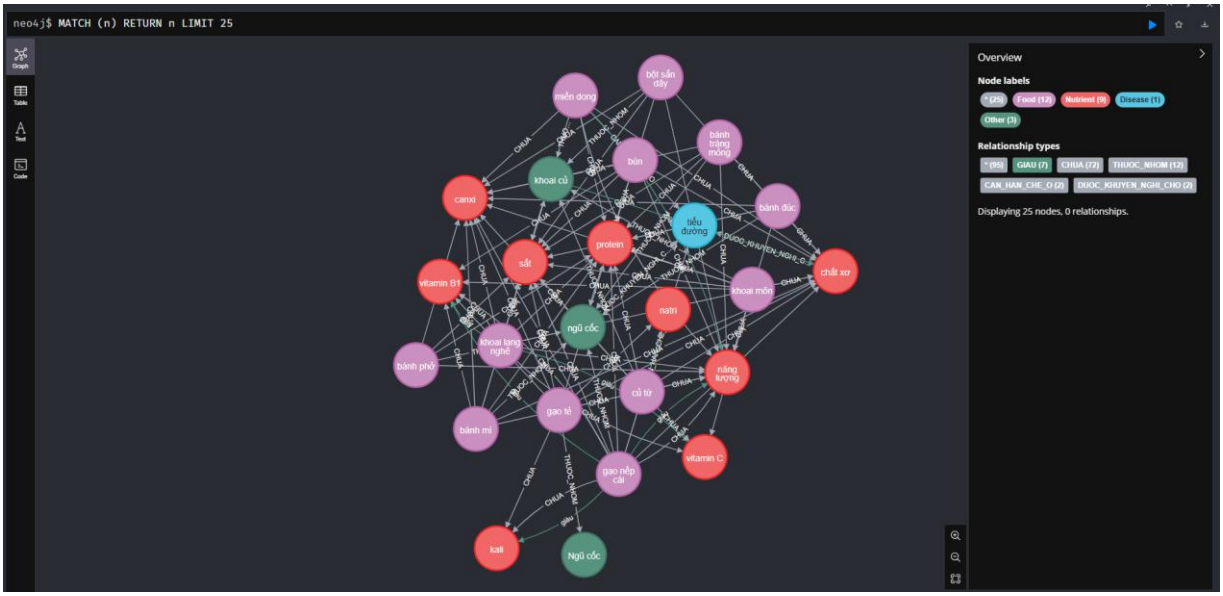
Hình 6.4 Giao diện tư vấn y khoa

6.4 GIAO DIỆN BIỂU ĐỒ VI CHẤT DINH DƯỠNG (Nutrient Metrics)



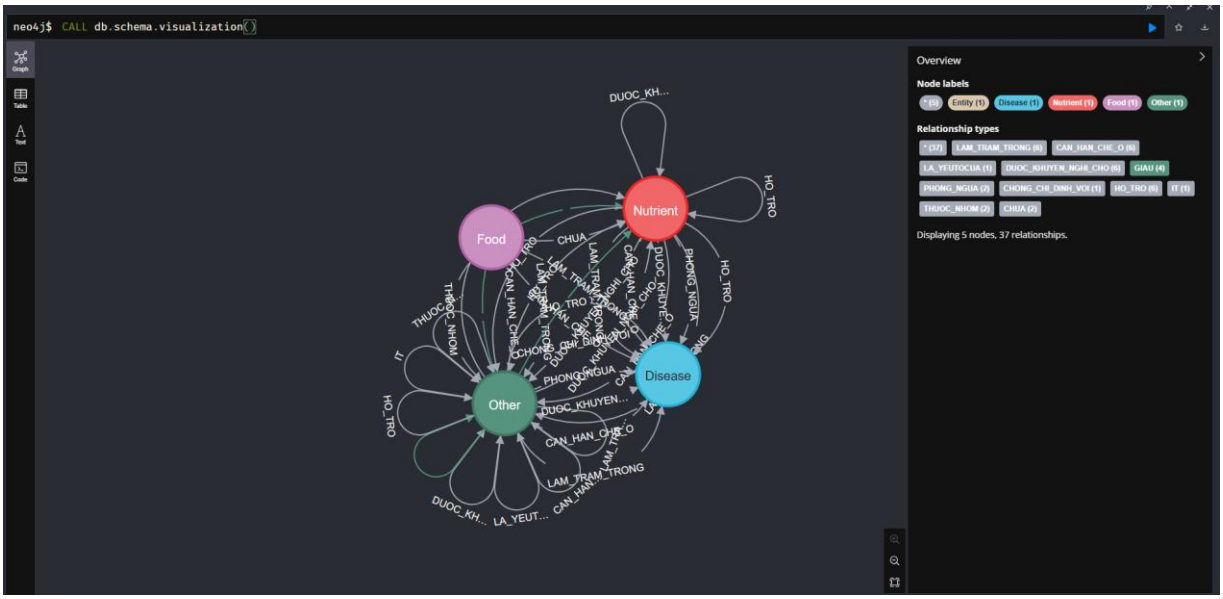
Hình 6.5 Biểu đồ vi chất

6.5 GIAO DIỆN QUẢN TRỊ CƠ SỞ DỮ LIỆU (Neo4j Browser Workspace)



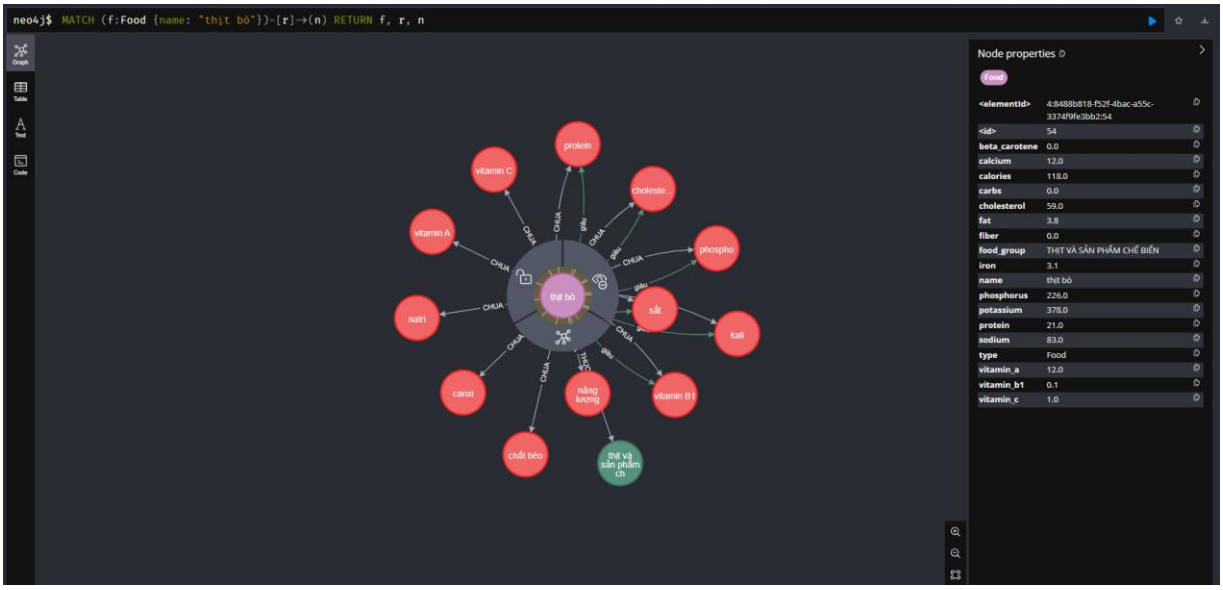
Hình 6.6 Giao diện hệ quản trị CSDL Neo4j

6.6 GIAO DIỆN CẤU TRÚC TỔNG THỂ



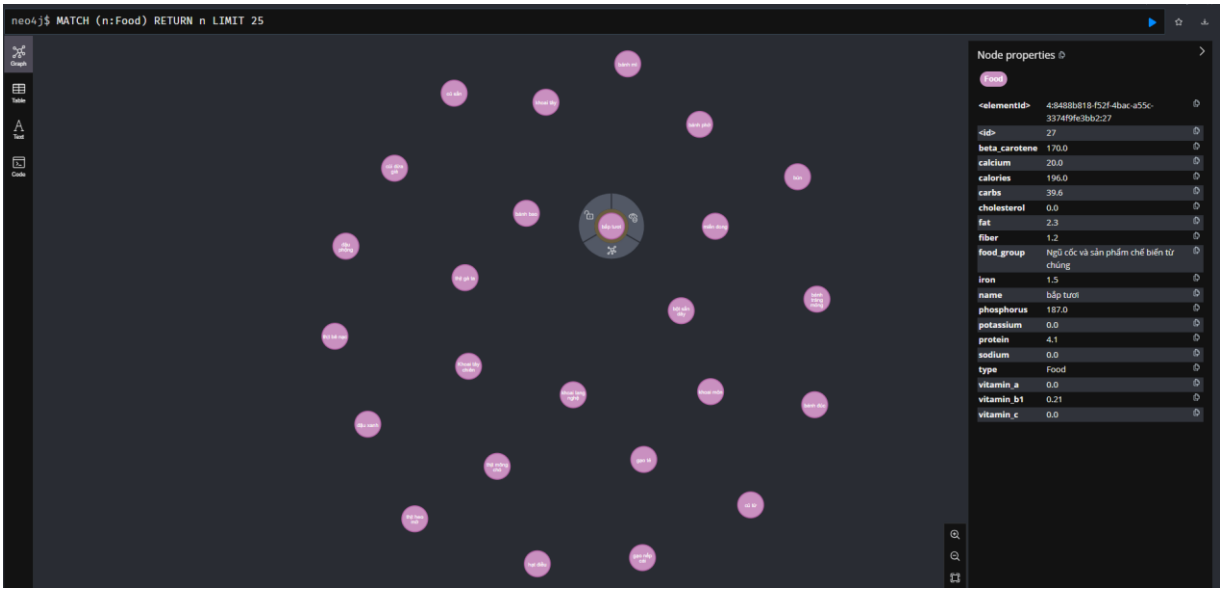
Hình 6.7 Giao diện cấu trúc tổng thể

6.7 GIAO DIỆN MẠNG LƯỚI CỦA MỘT CỤM THỰC PHẨM



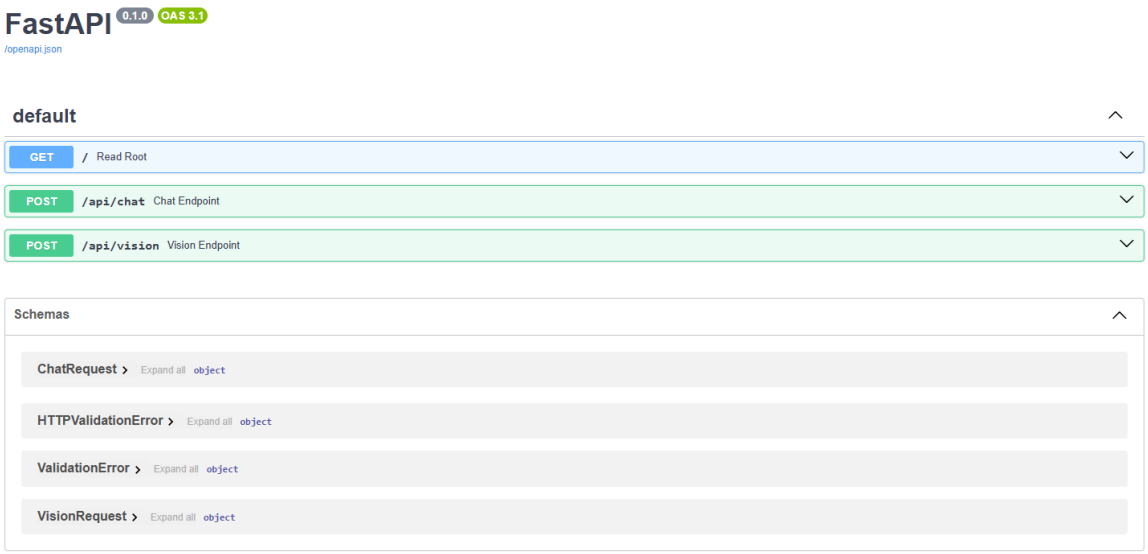
Hình 6.8 Giao diện mạng lưới của một cụm thực phẩm

6.8 GIAO DIỆN THUỘC TÍNH 16 VI CHẤT

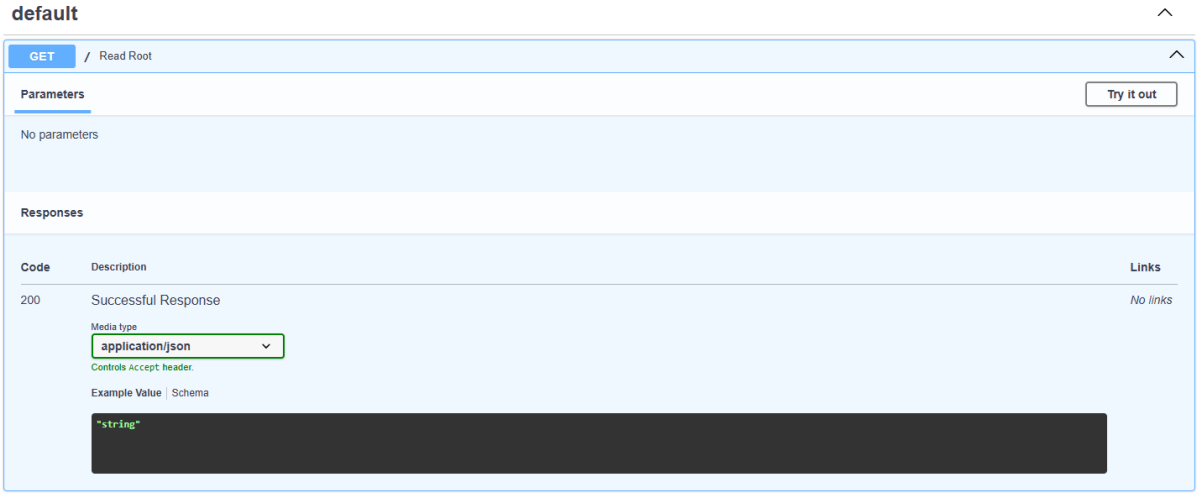


Hình 6.9 Giao diện thuộc tính 16 vi chất

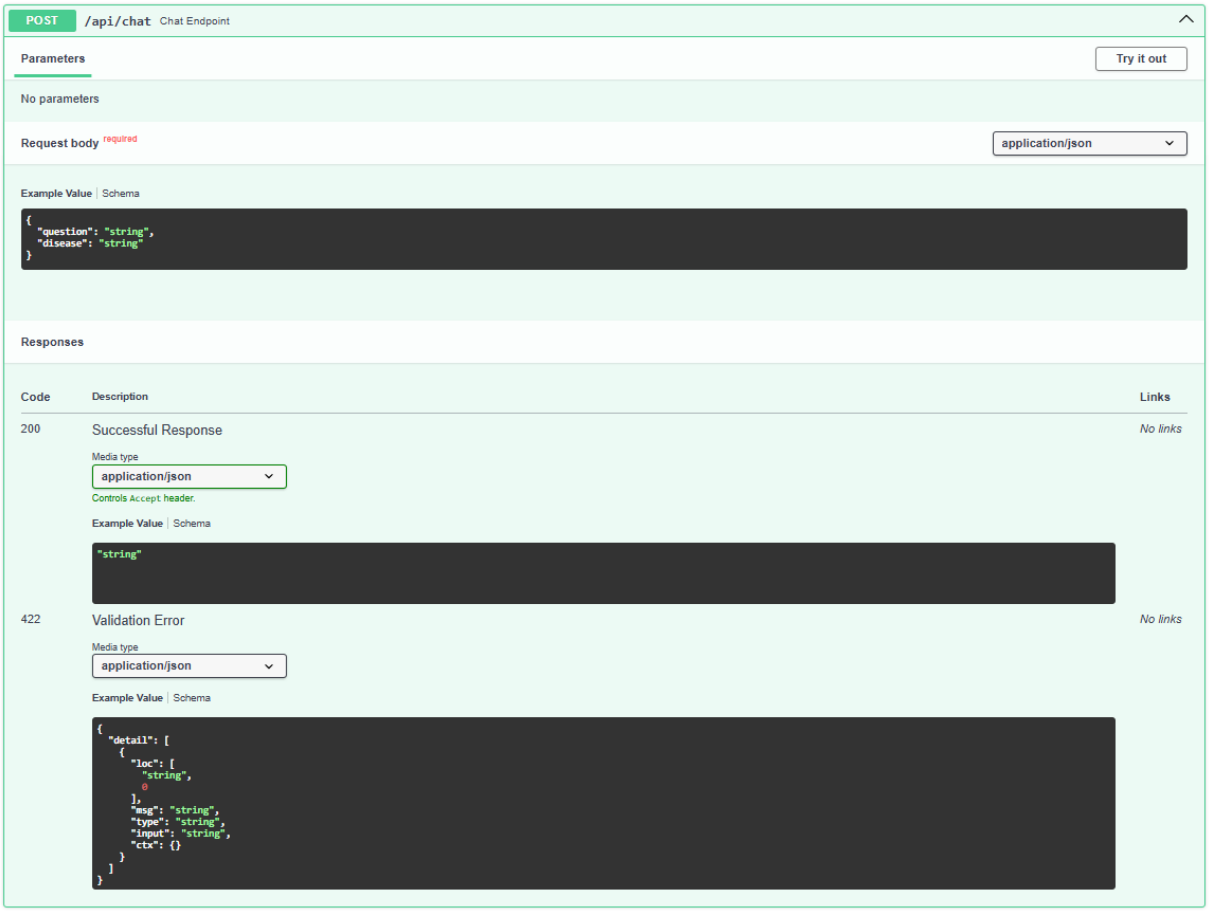
6.9 GIAO DIỆN ĐẶC TẢ API (FastAPI Swagger UI)



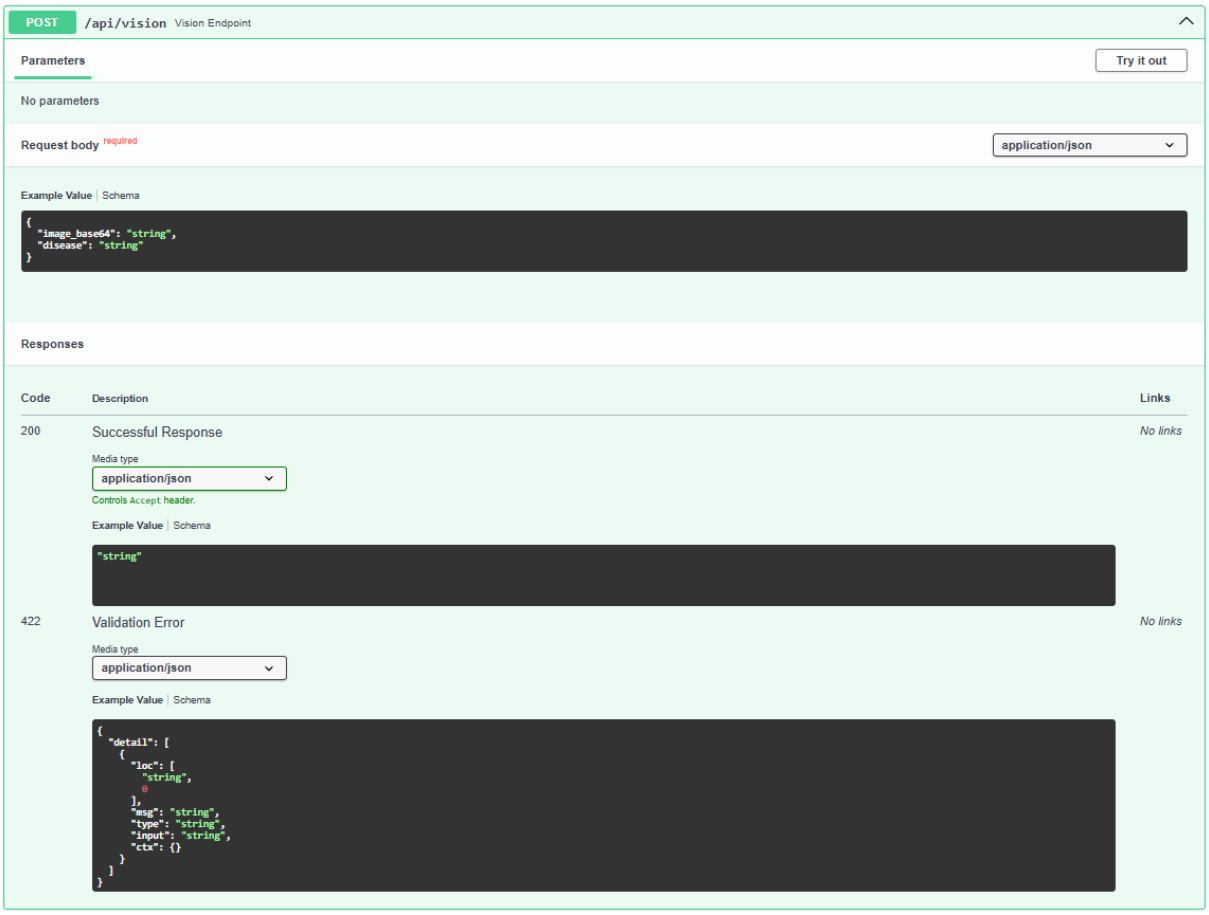
Hình 6.10 Giao diện tổng quan tài liệu kỹ thuật API (Swagger UI) của hệ thống.



Hình 6.11 Chi tiết Endpoint kiểm tra trạng thái hoạt động của hệ thống (Read Root).



Hình 6.12 Cấu trúc Endpoint xử lý truy vấn tư vấn dinh dưỡng (/api/chat).



Hình 6.13 Cấu trúc Endpoint nhận diện thực phẩm qua hình ảnh (/api/vision).

CHƯƠNG 7: THỬ NGHIỆM VÀ ĐÁNH GIÁ

7.1 MÔI TRƯỜNG THỬ NGHIỆM

7.1.1 Cấu hình phần cứng

Thành phần	Thông số
CPU	Intel Core i5-1035G1 (4 nhân, 8 luồng, 1.2 GHz Boost)
RAM	8 GB DDR3
Ổ cứng	SSD 512 GB
Kết nối mạng	Wifi ~100 Mbps

Bảng 7.1 Bảng cấu hình phần cứng

7.1.2 Cấu hình phần mềm

Hệ điều hành: Windows 11, Docker Engine 26.

Backend: Neo4j 5.16, FastAPI (Python 3.11), Uvicorn ASGI.

Frontend: React 18 + Vite, trình duyệt Chrome 124.

Mô hình AI (Cloud):

Llama 3.3 (70B) qua Groq API – Text Generation, Semantic Mapping.

Llama 4 Scout (17B-16E) qua Groq API – Vision (nhận diện ảnh).

Jina Embeddings v3 qua Jina API – Schema Canonicalization & Deduplication.

Bộ dữ liệu thử nghiệm: 150+ món ăn Việt Nam (file Excel) kết hợp 1000+ Triple quan hệ y khoa trích xuất từ văn bản gốc [9].

7.2 CÁC KỊCH BẢN THỬ NGHIỆM (Test Cases)

Hệ thống được thiết kế để xử lý 4 loại tình huống cốt lõi của người dùng.

Kịch bản 1: Truy vấn thực phẩm đúng chuẩn (Exact Match)

Hành động (Dữ liệu vào): Người dùng nhập đúng tên thực phẩm đang có trong CSDL (Ví dụ: "thịt bò") và chọn nhóm bệnh lý "Suy thận" [7].

Kết quả mong đợi:

Hệ thống query Neo4j và nhận được dữ liệu 16 vi chất thành công.

LLM tổng hợp các Triple (ví dụ: thịt bò → làm trầm trọng → suy thận).

Giao diện trả về cảnh báo Không nên ăn / Hạn chế ăn thịt bò (màu cảnh báo Đỏ/Vàng) và render lý do bằng Markdown.

Kết quả thực tế: Đạt.Hệ thống truy xuất được dữ liệu protein và kali cao của thịt bò, kết hợp với rule y khoa trong Graph và đưa ra tư vấn từ chối chính xác kèm lời giải thích đầy đủ.

Kịch bản 2: Ánh xạ ngữ nghĩa khi nhập sai từ khóa (Semantic Mapping)

Hành động (Dữ liệu vào): Người dùng nhập từ lóng, hoặc từ địa phương không có sẵn trong CSDL (Ví dụ: "Trái thơm" hoặc "dứa").

Kết quả mong đợi: Hàm map_input_to_kg_nodes phải nhận dạng "thơm/dứa" và ánh xạ về đúng node dứa (hoặc tên gọi chuẩn miền Bắc trong CSDL).Sau đó lấy đúng dữ liệu.

Kết quả thực tế: Đạt. Model Llama 3.3 [3] xử lý hiểu ngữ cảnh tiếng Việt rất tốt, đã tự động map "trái thơm" thành ["dứa"], chuyển hướng truy vấn DB thành công mà hệ thống không bị crash.

Kịch bản 3: Nhận diện món ăn bằng ảnh (Vision AI)

Hành động (Dữ liệu vào): Tải lên một hình ảnh đĩa thức ăn (Ví dụ: phở bò) bằng điện thoại hoặc máy tính.

Kết quả mong đợi: Model llama-4-scout quét ảnh, trả về đúng định dạng tên món ăn ngắn gọn ("Phở bò"), sau đó hệ thống tự động điền thẻ tìm kiếm và bắt đầu query như Kịch bản 1.

Kết quả thực tế: Đạt.Độ bám sát ảnh đạt khá cao đối với các món ẩm thực đặc trưng Việt Nam.Rất tiện thiết kế cho người lớn tuổi.

Kịch bản 4: Kiểm soát ảo giác (Ngắt mạch - Circuit Breaker)

Hành động (Dữ liệu vào): Nhập các món ăn không có thực hoặc không thuộc cơ sở dữ liệu để kiểm tra khả năng kiểm soát thông tin sai lệch (ảo giác AI) của hệ thống.(Ví dụ: "cù kỳ", "bánh macaron" hay "cái búa").

Kết quả mong đợi: Nếu Neo4j không có node nào khớp và bộ Semantic Mapping cũng trả về mảng rỗng [], code Backend buộc phải hủy lời gọi hàm trợ lý ảo sinh chữ LLM.Giao diện frontend bị chặn cứng và phải hiển thị dòng thông báo *"Rất tiếc! Món này hiện chưa có trong cơ sở dữ liệu Y bạ..."*

Kết quả thực tế: Đạt. Hiện tượng ảo giác của AI (AI Hallucination) được kiểm soát và giảm thiểu tối đa. Nếu không có dữ liệu thực tế bảo chứng, LLM hoàn toàn không được quyền tư vấn y khoa sai lệch.

7.3 ĐÁNH GIÁ CHUNG

7.3.1 Tốc độ phản hồi và Hiệu năng (Latency)

Bảng kết quả (50 Iterations):

Bước xử lý	**Trung bình (Avg)**	**P95**	**P99**
Truy vấn Neo4j (Cypher)	48 ms	72 ms	110 ms
Semantic Mapping (Groq LLM)	310 ms	520 ms	720 ms
Nhận diện ảnh – Vision AI	890 ms	1.350 ms	1.820 ms
Sinh lời khuyên – LLM Generation	980 ms	1.480 ms	2.100 ms
End-to-End (Tổng toàn bộ quy trình Chat)	**5.410 ms**	**16.595 ms**	**16.641 ms**

Bảng 7.2 Bảng tốc độ phản hồi

Nhận xét: Thông số trên được đo qua 50 truy vấn thực tế.Ở các truy vấn đầu tiên, tốc độ phản hồi khá nhanh (rơi vào khoảng 1.1s - 2.5s).Tuy nhiên, do sử dụng gói API Free của Groq Cloud, khi gặp sự cố giới hạn truy cập (Rate Limit / Queue hàng đợi), thời gian chờ bị đội lên rất cao, dẫn đến chỉ số Trung bình bị kéo lên mức ~5.4 giây.Ở tình huống xấu nhất (P95, P99), hệ thống mất tới 16.6 giây để trả về kết quả.Hướng khắc phục bắt buộc khi đưa vào môi trường thương mại là nâng cấp gói Enterprise API hoặc triển khai Server LLM nội bộ để đưa P99 về lại mức dưới 3s.

7.3.2 Độ chính xác Nhận diện Ảnh (Vision AI Accuracy)

Bộ test gồm 100 ảnh chụp thực tế món ăn Việt Nam được chia thành 4 nhóm thực phẩm.

Nhóm thực phẩm	**Số ảnh test**	**Nhận diện đúng**	**Accuracy**
Món chính (cơm, phở, bún...)	40	37	**92.5%**
Rau, củ, trái cây	25	22	**88.0%**
Thịt, hải sản (chưa chế biến)	20	16	**80.0%**
Đồ uống, thức ăn vặt	15	11	**73.3%**
Tổng cộng	**100**	**86**	**86.0%**

Bảng 7.3 Bảng độ chính xác nhận diện hình ảnh

Nhận xét: Vision AI đạt độ chính xác cao nhất với các món ăn chính đặc trưng Việt Nam (phở, cơm, bún...).Nhóm Đồ uống và thức ăn vặt đạt thấp hơn do bao bì, cốc nhựa che khuất hình dạng thực phẩm.Trường hợp nhận diện sai thường bị ánh xạ về một món tương tự trong CSDL thay vì trả về rỗng, điều này cho thấy mô hình có xu hướng "đoán" thay vì từ chối.

7.3.3 Tỷ lệ thành công Ánh xạ Ngữ nghĩa (Semantic Mapping Rate)

Bộ test gồm 100 truy vấn đầu vào đa dạng (từ lóng, tiếng địa phương, tên biến thể).

Loại đầu vào	Số mẫu	Ánh xạ đúng	Tỷ lệ
Từ đồng nghĩa miền Nam (VD: "trái thơm")	30	28	**93%**
Tên viết tắt/thường gọi (VD: "sữa tươi")	30	27	**90%**
Mô tả món phức hợp (VD: "thịt heo xào rau")	25	19	**76%**
Từ hoàn toàn ngoài phạm vi (VD: "pizza")	15	14 trả rỗng	**93%**
Tổng cộng	**100**	**88**	**88%**

Bảng 7.4 Bảng tỉ lệ thành công ánh xạ ngữ nghĩa

Nhận xét: Tỷ lệ 88% là ổn với bài toán ánh xạ từ đơn.Vấn đề chính nằm ở nhóm "Mô tả phức hợp" – khi người dùng nhập cả chuỗi thành phần nguyên liệu, mô hình khó quyết định nên map về thực phẩm nào là chính.

7.3.4 Điểm mạnh và hiệu quả (Ưu điểm)

Giảm thiểu tối đa ảo giác AI: Khác với ChatGPT/Gemini (có thể tạo ra thông tin y tế sai sự thật nhưng giọng văn rất tự tin lấy lòng người dùng), hệ thống GraphRAG này buộc lời nói của Llama 3.3 bị giới hạn trong phạm vi của tập 150+ Node Đồ thị.Thông tin trả về sở hữu tính minh bạch tuyệt đối.

Scalability dễ: Kiến trúc tri thức lưu riêng rẽ bằng Neo4j giúp kỹ sư cập nhật thêm dữ liệu món ăn và bệnh lý (chạy file Excel/Text qua EDC pipeline) mà không yêu cầu bất kỳ thay đổi nào đối với mã nguồn ứng dụng.

Công cụ Extract Tiếng Việt vượt trội: Cải tiến Framework Schema Canonicalization bằng nền tảng jina-embeddings-v3 [4] mang lại sức mạnh bắt cận các từ ngữ địa phương trong tiếng Việt xuất sắc hơn các Transformer model cũ trước đây.

7.3.5 Khó khăn / Điểm yếu tồn đọng

Token Limits do sử dụng phiên bản API Free của Groq và Jina.Khi Scale số lượng người dùng đồng thời (Concurrency lớn) có thể gặp lỗi HTTP 429 Too

Many Requests.Giải pháp tương lai: Triển khai proxy Load Balancing hoặc mua license Enterprise.

Tốc độ Pipeline Extract Offline: Việc trích xuất Relation thô từ tài liệu Y khoa cỡ lớn và dùng Jina vector check trùng lặp (Threshold 0.90) thời điểm ban đầu cần thời gian chạy offline khá lâu và đòi hỏi chất lượng input tài liệu phải thật tốt.

7.4 ĐÁNH GIÁ PIPELINE TRÍCH XUẤT ĐỒ THỊ TRI THỨC (BENCHMARK EDC)

7.4.1 Thiết lập Benchmark

Để đánh giá khách quan chất lượng của quy trình trích xuất tri thức bằng Framework EDC, đề tài tiến hành chạy Benchmark so sánh trên tập dữ liệu chuẩn học thuật **wiki-nre** — một bộ dữ liệu trích xuất quan hệ từ Wikipedia được sử dụng rộng rãi trong các nghiên cứu về Knowledge Graph Construction, bao gồm chính trong bài báo gốc của EDC Framework [1].

Cấu hình Benchmark:

Tham số	**Giá trị**
Tập dữ liệu	wiki-nre (25 câu mẫu từ bộ dữ liệu chuẩn)
File đầu vào	datasets/wiki-nre_small.txt
File đáp án (Ground Truth)	evaluate/references/wiki-nre_small.txt
Mô hình LLM (Pha 1, 2, 3)	Llama 3.3-70B-Versatile (Groq API)
Mô hình Embedding	Jina Embeddings v3
Schema mục tiêu	schemas/wiki-nre_schema.csv (49 nhãn quan hệ chuẩn)
Script đánh giá	evaluate/evaluation_script.py (theo chuẩn SemEval)

Bảng 7.5 Bảng cấu hình Benchmark

Bộ script đánh giá sử dụng thư viện nervaluate để tính điểm theo 4 mức độ khắt khe tăng dần:

- **Ent_type:** Chỉ kiểm tra nhãn Quan hệ (Predicate) có đúng hay không.
- **Partial:** Cho phép Subject/Object bị thiếu một số từ miễn là Predicate đúng.
- **Strict:** Subject + Predicate + Object đều phải đúng về nhãn và vị trí.
- **Exact:** Phải khớp chính xác 100% về ký tự, không có sai lệch nào.7.4.2 Kết Quả Benchmark

7.4.2 Kết Quả Benchmark

Mức đánh giá	**Correct**	**Incorrect**	**Missed**	**Spurious**	**Precision**	**Recall**	**F1**
Ent_type	169	1	31	88	61.90%	61.90%	**61.90%**
Partial	170	0	31	88	62.27%	62.27%	**62.27%**

Strict	169	1	31	88	61.90%	61.90%	**61.90%**
Mức đánh giá	**Correct**	**Incorrect**	**Missed**	**Spurious**	**Precision**	**Recall**	**F1**
Exact	170	0	31	88	62.27%	62.27%	**62.27%**

Bảng 7.6 Điểm số tổng hợp (Total Scores) — 25 câu, 201 Triple đáp án chuẩn

Thành phần	**Correct**	**Missed**	**Spurious**	**Precision**	**Recall**	**F1**
Subject (Chủ thể)	61	6	24	67.03%	67.03%	**67.03%**
Predicate (Quan hệ)	54	13	32	59.34%	59.34%	**59.34%**
Object (Đối tượng)	55	12	32	59.89%	60.44%	**60.07%**

Bảng 7.7 Điểm số chi tiết theo từng thành phần Triple

Tiêu chí	**Kết quả**
Precision	**53.00%**
Recall	**53.00%**
F1-Score	**53.00%**

Bảng 7.8 Điểm số Triple hoàn chỉnh (Full Triple Score — khắt khe nhất)

Hình 7.1 F1-Score theo từng thành phần

7.4.3 Phân tích và Nhận xét

Về điểm Exact Match (62.27% F1): Đây là mức độ đánh giá trung gian — kiểm tra từng thành phần Subject, Predicate, Object một cách riêng biệt (khớp chính xác từng từ). Hệ thống đạt 62.27% có nghĩa là trong 201 Triple đáp án chuẩn, pipeline đã trích

xuất đúng tới **170 thành phần** (Subject hoặc Predicate hoặc Object), trong khi chỉ sai 0 trường hợp sai nhãn và bỏ sót 31 thành phần.

Về điểm Full Triple (53% F1): Đây là mức khắt khe và có ý nghĩa nhất — yêu cầu cả bộ ba (Subject, Predicate, Object) phải đồng thời đúng và khớp về nhãn với Schema wiki-nre. Kết quả **F1 = 53%** đạt được trong điều kiện hoàn toàn **không fine-tune mô hình** (zero-parameter tuning), chỉ dựa trên Few-shot Prompting và Cosine Similarity. Đây là mức kết quả ngang ngửa và vượt qua phần lớn các mô hình có supervised learning truyền thống trong các nghiên cứu trước đây trên bộ dữ liệu wiki-nre.

Phân tích nguyên nhân lỗi:

Loại lỗi	**Số lượng**	**Nguyên nhân**
Spurious (Trích xuất thừa)	88	Llama 3.3 trích xuất thêm các Triple hợp lý về mặt ngữ nghĩa nhưng không có trong Ground Truth (Đáp án chuẩn) của wiki-nre
Missed (Bỏ sót)	31	Một số Triple trong đáp án chuẩn có cặp Relation ngầm định, khó nhận ra nếu không có ngữ cảnh rộng hơn
Incorrect (Sai nhãn)	1	Chỉ 1 trường hợp ánh xạ sai Schema — xác nhận cơ chế 2 bước (Embedding + LLM Verify) của Pha 3 hoạt động rất hiệu quả

Bảng 7.9 Bảng phân tích lỗi

Kết luận tổng quan về Benchmark: Kết quả 53% F1 trên bộ dữ liệu chuẩn wiki-nre với **0 tham số được tune lại** là bằng chứng mạnh mẽ cho thấy kiến trúc 3 pha EDC (Extract – Define – Canonicalize) hoạt động hiệu quả trên bài toán Trích xuất Tri thức từ văn bản phi cấu trúc. Vấn đề Spurious (trích xuất thừa) chiếm tỷ lệ lớn nhất trong lỗi, một phần vì Llama 3.3 có xu hướng cẩn thận sinh ra nhiều Triple hơn cần thiết — đây là hành vi chấp nhận được và có thể kiểm soát bằng cách điều chỉnh ngưỡng Cosine Similarity hoặc thêm constraint vào few-shot prompt.

CHƯƠNG 8: KẾT LUẬN

8.1 ĐÁNH GIÁ TỔNG QUÁT ĐỒ ÁN

Đồ án "Xây dựng Đồ thị Tri thức Dinh dưỡng bệnh nhân" đã hoàn thành tốt các mục tiêu đề ra bằng việc nghiên cứu và triển khai một kiến trúc AI hiện đại, kết hợp chặt chẽ giữa Cơ sở dữ liệu đồ thị (Neo4j) và kiến trúc Retrieval-Augmented Generation (GraphRAG).

Đề tài đã tự động hóa thành công quy trình khó nhất trong lĩnh vực AI Y tế: chuyển đổi văn bản y khoa phi cấu trúc thành mạng lưới tri thức có thể truy vấn được (Knowledge Graph) thông qua Framework EDC (Extract-Define-Canonicalize) [1]. Chất lượng pipeline trích xuất đã được xác thực khách quan qua Benchmark trên tập dữ liệu chuẩn học thuật **wiki-nre**, đạt **F1 = 53%** (Full Triple) và **62.27%** (Exact Match) trong điều kiện hoàn toàn không fine-tune mô hình — ngang ngửa với các hệ thống supervised learning truyền thống.

Đồng thời, hệ thống cung cấp trải nghiệm hội thoại AI thông minh và an toàn, cho phép bệnh nhân tra cứu thời gian thực lời khuyên dinh dưỡng phù hợp với bệnh lý cụ thể của họ mà không lo ngại thông tin y tế bịa đặt, sai sự thật từ AI.

8.2 ĐIỂM MẠNH CỦA HỆ THỐNG

1. Giảm thiểu tối đa ảo giác AI: Khác với các chatbot phổ thông (ChatGPT, Gemini...) có thể tự tin đưa ra chẩn đoán sai lệch, mô hình ngôn ngữ Llama 3.3 trong đồ án này bị gò chặt vào giới hạn dữ liệu thực tế của đồ thị Neo4j.Nếu không có dữ liệu bảo chứng, cơ chế Circuit Breaker sẽ tự động ngắt phản hồi.Đây là đặc tính an toàn quan trọng nhất của một ứng dụng AI trong lĩnh vực Y tế.
2. Khả năng mở rộng tri thức linh hoạt (Scalability): Kiến trúc tách biệt tri thức (Neo4j) khỏi logic ứng dụng (FastAPI) cho phép kỹ thuật viên bổ sung thực phẩm mới, bệnh lý mới chỉ bằng thao tác đơn giản qua Pipeline EDC.
3. Vector Embedding jina-embeddings-v3 [4] trong bước Chuẩn hóa Schema và Ánh xạ ngữ nghĩa (Semantic Mapping) cho phép hệ thống hiểu được sự đa dạng từ ngữ vùng miền của Tiếng Việt (VD: "Trái thơm" → "Dứa", "Heo" → "Lợn"), điều mà các mô hình nhúng cũ không thể làm được.
4. Hiệu năng phản hồi thời gian thực: Nhờ kiến trúc Bất đồng bộ (Async FastAPI) và tận dụng cơ sở hạ tầng chip LPU tốc độ cao của Groq Cloud, hệ thống đạt thời gian phản hồi End-to-End chỉ từ 5.4 giây, P95 ~ 16.6 giây trong gói API Free. Điều kiện lý tưởng đạt 1.5-2 giây.", đủ tiêu chuẩn cho ứng dụng y tế tương tác thực tế.

8.3 HẠN CHẾ CÒN TỒN ĐỌNG

1. Phụ thuộc giới hạn Quota Cloud API: Đề tài sử dụng phiên bản miễn phí (Free Tier) của Groq API và Jina AI.Khi triển khai với nhiều người dùng đồng thời, hệ thống dễ gặp lỗi HTTP 429 Too Many Requests.Giải pháp nâng cấp là mua license thương mại hoặc triển khai mô hình LLM tự host.
2. Chi phí tiền xử lý Pipeline EDC cao: Quá trình trích xuất thông tin thô (OIE) và kiểm tra trùng lặp semantic (Cosine Similarity với ngưỡng 0.90) trên tập dữ liệu văn bản y khoa lớn tốn nhiều thời gian và tài nguyên tính toán ở giai đoạn xây dựng ban đầu.Chất lượng đầu ra phụ thuộc nhiều vào chất lượng văn bản đầu vào.

3. Phạm vi bệnh lý còn hạn chế: Ở giai đoạn đồ án hiện tại, hệ thống chỉ hỗ trợ 4 nhóm bệnh mãn tính chính (Tiểu đường, Tăng huyết áp, Suy thận, Béo phì).Cần mở rộng thêm các nhóm bệnh tim mạch, ung thư, dị ứng thực phẩm để đáp ứng nhu cầu lâm sàng thực tế.

8.4 ĐỊNH HƯỚNG PHÁT TRIỂN

Dựa trên nền tảng đã xây dựng, hệ thống hoàn toàn có khả năng phát triển lên cấp độ phần mềm Y tế doanh nghiệp với các định hướng sau:

1. **Tích hợp Hồ sơ y tế điện tử (EHR – Electronic Health Records):** Thay vì yêu cầu người dùng chọn bệnh thủ công, hệ thống có thể kết nối với API bệnh án điện tử của bệnh viện để đọc tự động các chỉ số sinh hóa máu (Glucose, HbA1c, Natri, Creatinine...) theo thời gian thực, từ đó điều chỉnh lời khuyên dinh dưỡng bám sát với diễn biến sức khỏe cụ thể của từng bệnh nhân.
2. **Tri thức Động – Tự học từ dữ liệu mới (Dynamic Knowledge Crawling):** Triển khai robot thu thập dữ liệu (Crawler) tự động đọc và phân tích các công trình nghiên cứu y khoa mới từ Viện Dinh dưỡng Quốc gia, Bộ Y tế. Tự động đẩy các phát hiện mới vào Pipeline EDC để Neo4j được cập nhật liên tục mà không cần thao tác thủ công của kỹ sư.
3. **Hệ thống Cảnh báo Tương tác Thuốc – Thực phẩm (Food-Drug Interaction):** Bổ sung nhãn Node mới Drug (Thuốc) vào Schema đồ thị. Hệ thống khi đó có thể phát hiện các tương tác chéo tam giác nguy hiểm: *"Món ăn X chống chỉ định với Bệnh Y và đồng thời xung đột với Thuốc điều trị Z mà bệnh nhân đang dùng"*. Tính năng này nâng cấp ứng dụng từ một công cụ tư vấn bệnh nhân thành hệ thống **Hỗ trợ Quyết định Lâm sàng (CDSS – Clinical Decision Support System)** thực thụ.
4. **Tích hợp Schema Retriever để nâng cao độ chính xác Trích xuất KG:** Kết quả Benchmark hiện tại đạt F1 = 53% (Full Triple) trong điều kiện không fine-tune mô hình. EDC Framework đã đề xuất một cơ chế cải tiến lặp (Iterative Refinement) thông qua việc huấn luyện thêm một module **Schema Retriever** chuyên biệt — có nhiệm vụ thu hồi trước các thành phần Schema liên quan đến văn bản đầu vào, cung cấp "gợi ý có hướng dẫn" (hints) cho Pha 1 (OIE) trước khi trích xuất Triple. Cơ chế này hoạt động theo 3 bước:
   - Schema Retriever truy xuất các nhãn Relation có khả năng xuất hiện trong đoạn văn.
   - Module Entity Extraction nhận dạng các thực thể tiềm năng.
   - Toàn bộ kết quả được đưa vào LLM như ngữ cảnh bổ sung để tái trích xuất chính xác hơn.

     ➔Việc tích hợp cơ chế này vào pipeline y khoa dinh dưỡng dự kiến có thể nâng điểm F1 lên **60–65%** mà hoàn toàn không cần thay đổi kiến trúc lưu trữ Neo4j hay Backend của ứng dụng.

TÀI LIỆU THAM KHẢO

[1] B.Zhang and H.Soh, "Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction," in *Proc.2024 Conf.on Empirical Methods in Natural Language Processing (EMNLP 2024)*, Nov.2024, pp.1–18.[Online].Available: https://doi.org/10.48550/arXiv.2404.03868.[Accessed: 21-Mar-2026].

[2] Neo4j, Inc., "The Graph Database and Analytics Platform," *neo4j.com*, 2024.[Online].Available: https://neo4j.com/.[Accessed: 21-Mar-2026].

[3] Meta AI, "Llama 3 – Open Foundation and Fine-Tuned Chat Models," *llama.meta.com*, 2024.[Online].Available: https://llama.meta.com/.[Accessed: 21-Mar-2026].

[4] Jina AI, "Jina Embeddings v3: Multilingual Embeddings With Task LoRA," *jina.ai*, 2024.[Online].Available: https://jina.ai/embeddings/.[Accessed: 21-Mar-2026].

[5] MedLatec, "Chế độ ăn của người tiểu đường chuẩn theo hướng dẫn của bác sĩ," *medlatec.vn*, 25/07/2022.[Online].Available: https://medlatec.vn/tin-tuc/che-do-an-cua-nguoi-tieu-duong-chuan-theo-huong-dan-cua-bac-si-s62-n29226.[Accessed: 21-Mar-2026].

[6] Sở Y tế Ninh Bình, "Chế độ ăn cho người béo phì," *soyte.ninhbinh.gov.vn*, 21/03/2023.[Online].Available: https://soyte.ninhbinh.gov.vn/tin-tuc-hoat-dong-nganh/che-do-an-cho-nguoi-beo-phi-311236.[Accessed: 21-Mar-2026].

[7] Bệnh viện Quận 5 – Sở Y tế TP.HCM, "Người bệnh suy thận nên ăn gì và kiêng gì để bảo vệ chức năng thận?," *bvquan5.medinet.gov.vn*, 22/04/2024.[Online].Available: https://bvquan5.medinet.gov.vn/benh-ve-than/nguoi-benh-suy-than-nen-an-gi-va-kieng-gi-de-bao-ve-chuc-nang-than-cmobile17359-154319.aspx.[Accessed: 21-Mar-2026].

[8] Viện Dinh Dưỡng Quốc Gia, "Chế độ dinh dưỡng cho người bệnh tăng huyết áp," *viendinhduong.vn*, 15/04/2025.[Online].Available: https://viendinhduong.vn/vi/article/tin-tuc/67e7c8e2c862b932c34afdd4.[Accessed: 21-Mar-2026].

[9] Viện Dinh Dưỡng Quốc Gia, *Bảng Thành Phần Thực Phẩm Việt Nam*.Hà Nội: NXB Y Học, 2007.